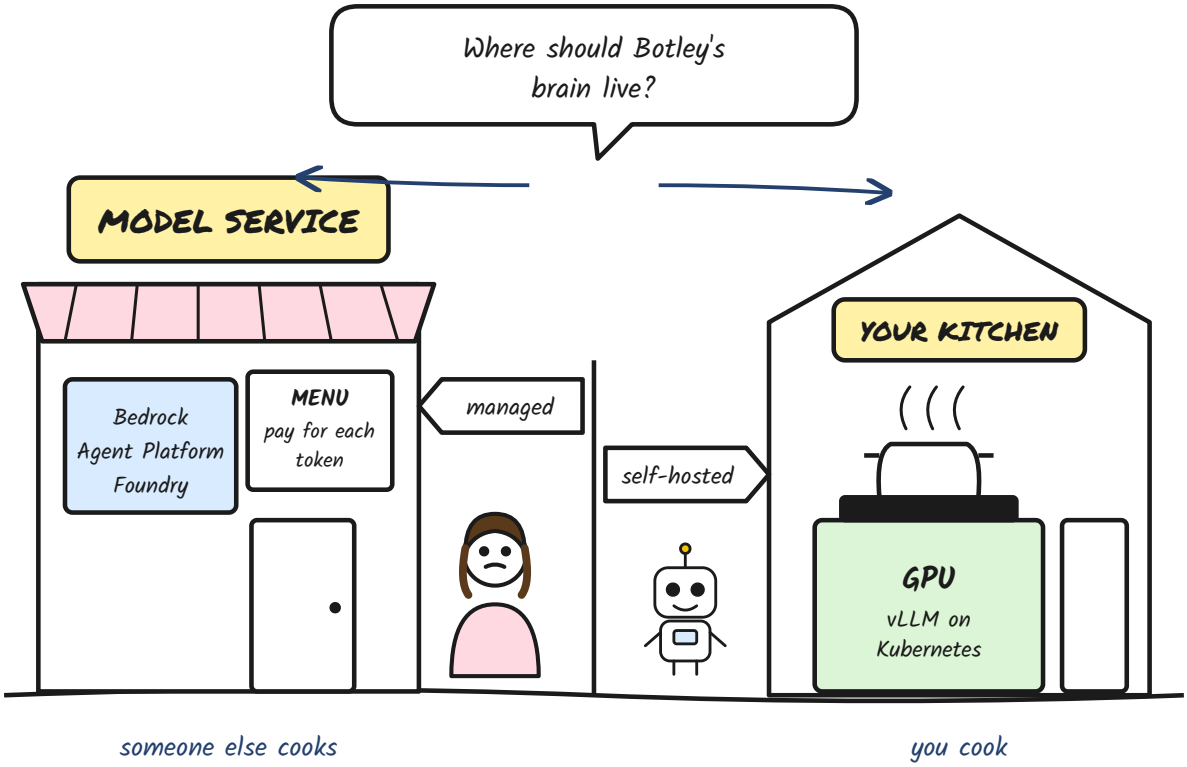
A visual notebook · Ship It with Nimesha #2

Running Your Own Models

Real prices, real commands

Managed model services or your own GPUs? How each works, and when each one pays off.



Learn what a model really is, how it uses GPU memory, what Amazon Bedrock, Google's Agent Platform and Microsoft Foundry offer, and how to serve an open model with vLLM on Kubernetes. Then do the math to see when running your own is worth it.

Will my model fit on this GPU?

Bedrock, Agent Platform and Foundry side by side

vLLM on GKE, EKS and AKS, step by step

Nimesha Jinarajadasa

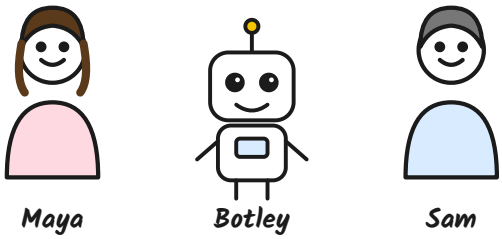
First edition, October 2026

How to use this notebook

Pictures, stories and real numbers. No experience with GPUs needed.

This notebook is for people who need to decide where an AI model should run. Maybe you are a developer whose agent works well and now costs more every month. Maybe you are a manager who has heard "we should just run our own model" and wants to know what that really means. Or maybe you keep hearing words like *GPU*, *vLLM* and *tokens per second* and want them explained simply.

You don't need to have read Notebook #1, but it helps to know what an AI agent is. The same three characters are back:



Maya builds agents. **Botley** is the agent she built, and it is now live. **Sam** is her boss, who has two questions: is the model bill too high, and can a customer's data stay inside the company's own cloud account?

One picture for the whole notebook

Think of the AI model as food that Botley needs many times a day.

In the kitchen	In the cloud
Eating at a restaurant	A shared model service. You pay for each token.
Booking a private room	Reserved capacity on a model service, paid by the hour.
Cooking in your own kitchen	Your own GPUs, running a model you chose.
A secret recipe	A closed model, such as Claude, GPT or Gemini. Only available through an API.
A published recipe	An open-weight model, such as Llama, Qwen, Gemma, Mistral, gpt-oss or DeepSeek.
Counter space	GPU memory. The model and every conversation must fit on it.
The head chef	vLLM, the software that serves the model.

Boxes you will see

Box	What it is for
There are no silly questions	Questions people often want to ask but don't.
Think about it	Stop and guess the answer before you read on. It helps you remember.
Careful!	Mistakes that real teams make.
Your turn	A short exercise. The answers are at the back.
Face-off	Two people or ideas argue with each other.
Geek corner	A little bit of code or math. You can skip it.
Key takeaways	The main points of the chapter.

About the numbers

Every price, GPU size and command in this notebook was checked against the providers' own documents in October 2026, and each chapter shows the date its facts were checked. Prices and products change often, so check the latest documents before you decide.

What's inside

#	Chapter	The question it answers	Page
1	Where does Botley's brain live?	What does it mean to run your own model, and what are the other options?	3
2	What is a model, really?	What is inside a model, how big is it, and what may I do with it?	7
3	How a model writes an answer	What happens inside the GPU when Botley asks the model something?	10
4	Meet the GPU	Which GPU do I need, and will my model fit on it?	14
5	The managed path	What can the cloud's own model services do for us?	18
6	The do-it-yourself path	What software runs an open model on our own GPUs?	22
7	Models on Kubernetes	What changes when you run a model on Kubernetes instead of a normal app?	26
8	Deploying a model for real	What are the real steps to serve an open model on Kubernetes?	30
9	Does it pay off?	When is running your own model cheaper than paying per token?	38
10	Reasons that aren't money	When does running your own model make sense even if it costs more?	42
11	Choosing your path	So which path should Botley take?	45
	Back of the book	Word list and exercise answers.	48

Chapter one

1 Where does Botley's brain live?

The agent and the model are two different things. There are three ways to get a model.

Facts in this chapter last checked on 19 October 2026.



The model bill went up again. Can't we just run our own?

Botley is live. People use it every day to plan trips. Maya is happy. Then Sam comes in with two questions.

Question 1: *"The bill for the AI model grows every month. Would our own model be cheaper?"*

Question 2: *"A big customer asks if their data can stay inside our own cloud account. Can it?"*

Before Maya can answer, she needs to be clear about one thing: what exactly would "our own model" mean?

The agent is not the model

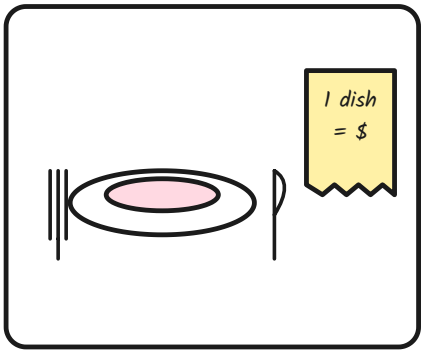
Botley is an **agent**. It is a program. It gets a goal, plans the steps, and uses tools such as flight search and the calendar. But Botley does not do the thinking itself. For every step, it sends text to a **large language model (LLM)** and gets text back. The model is the part that reads and writes.

Today, Botley sends that text over the internet to a model service. Botley's code runs in one place. The model runs somewhere else, on very powerful computers that someone else looks after.

In Notebook #1 we chose a home for the **agent**. This notebook is about a home for the **model**. They are separate choices. You can move the model without moving the agent.

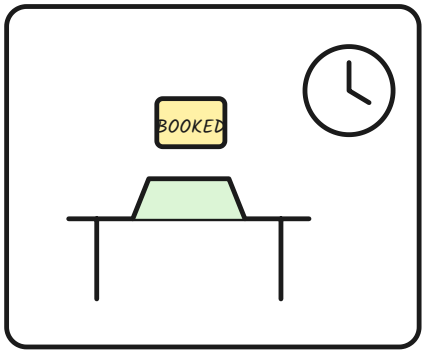
Three ways to get a meal

Think of the model as a meal that Botley needs many times a day. There are three ways to get it.



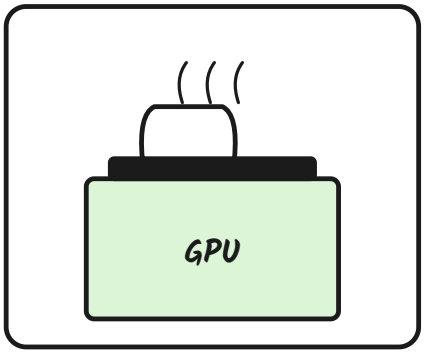
Eat at a restaurant

pay per token, shared service



Book a private room

reserved capacity, pay per hour



Cook in your own kitchen

your own GPUs, you run it all

	Eat at a restaurant	Book a private room	Cook in your own kitchen
In cloud words	A shared model service. You **pay per token**.	**Reserved capacity** on a model service.	**Your own GPUs**, running a model you chose.
How you pay	For each token that goes in and comes out.	By the hour, whether you use it or not.	For the GPU machines by the hour, plus your team's time.

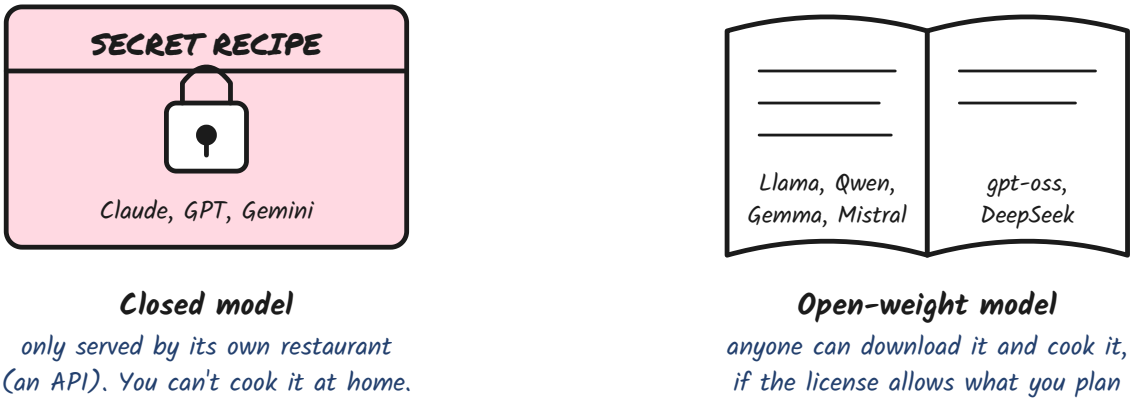
	Eat at a restaurant	Book a private room	Cook in your own kitchen
Who runs the computers?	The provider.	The provider.	You.
Which models?	The ones on the provider's list.	The ones on the provider's list.	Any open-weight model whose license fits your use.
Examples	Amazon Bedrock on-demand, Google's Agent Platform Standard PayGo, Microsoft Foundry Global Standard.	Bedrock Provisioned Throughput, Agent Platform Provisioned Throughput, Foundry Provisioned (PTU).	vLLM on Amazon EKS, Google GKE or Azure AKS.

A **token** is a small piece of text, often a short word or part of a word. Model services count everything in tokens, and they charge for the tokens you send and the tokens you get back. Chapter 3 explains tokens in more detail.

There is also a path between the restaurant and your own kitchen: the provider runs an open-weight model for you on GPUs that only you use. We look at it in Chapter 5.

Secret recipes and published recipes

Here is the most important fact in this chapter. **You can't cook every recipe at home.**



Closed model
only served by its own restaurant (an API). You can't cook it at home.

Open-weight model
anyone can download it and cook it, if the license allows what you plan

A **closed model** is like a secret recipe. The company that made it never gives out the model itself. You can only use it by sending requests to an API. Claude, GPT and Gemini are closed models. You can use them through their makers and through cloud services such as Bedrock, Agent Platform and Foundry. But you can't download them and run them on your own GPUs. (One exception: Google offers Gemini on its Google Distributed Cloud hardware, installed at a customer's own site.)

An **open-weight model** is like a published recipe. The model files (the "weights") are published, so anyone can download them and run them. Llama, Qwen, Gemma, Mistral, gpt-oss and DeepSeek are open-weight model families.

> ⚠ **CAREFUL!**
>
> **Running your own model usually means changing models.** If Botley uses Claude or GPT today, "our own model" means moving to an open-weight model. Its answers will be different. Some will be better, some worse. Before you switch, test the new model on real tasks. Chapter 11 shows how.

Think about it

Botley books flights, so it must call tools correctly every time. Would you be happy to switch Botley to a smaller open-weight model without testing it first? What could go wrong?

Open-weight is not the same as open source

People often say "open source model", but most of these models are only **open-weight**. You get the finished weights, but usually not the training data or the full training code. What you may do with the weights is set by the model's **license**. Some licenses, such as Apache 2.0, allow almost any use. Others have their own conditions. Chapter 2 shows how to check.

There are no silly questions

Q: Does "self-hosted" mean running the model on my laptop?

A: No. In this notebook it means running the model on GPU machines that you rent from a cloud, inside your own cloud account. You can run small models on a laptop to try them, but not for a live service with many users.

Q: Can I run Claude or GPT on my own GPUs if I pay extra?

A: No. Their makers don't give out the weights. You can only call them through an API. (OpenAI does publish a separate family of open-weight models, called gpt-oss. Those you can run yourself.)

Q: If an open-weight model is free to download, is it free to use?

A: The download is free. Running it is not. GPUs cost money every hour they are switched on, and someone has to look after them. Chapter 9 does the math.

Q: Can we use more than one way at the same time?

A: Yes, and many teams do. For example, a small model on your own GPUs for easy questions, and a large managed model for hard ones. Chapter 11 covers this.

Your turn

Which way is each of these: **pay per token**, **reserved capacity** or **your own GPUs**? One of them is not possible. Which one?

1. Botley calls Claude on Amazon Bedrock and the bill shows a price per million tokens.

2. Maya's team runs Qwen with vLLM on a GPU node pool in their GKE cluster.

3. The company pays Microsoft by the hour for Provisioned Throughput Units (PTUs), busy or not.

4. Sam asks Maya to download Gemini and run it on their EKS cluster.

The answers are at the back.

Key takeaways

- The agent and the model are separate. You can move the model without moving the agent.
- There are three ways to get a model: pay per token, reserved capacity, or your own GPUs.
- Closed models (Claude, GPT, Gemini) are only available through an API. You can't run them yourself.
- Open-weight models (Llama, Qwen, Gemma, Mistral, gpt-oss, DeepSeek) can be downloaded and run on your own GPUs.
- Running your own model usually means changing models, so test before you switch.
- Open-weight is not the same as open source. The license decides what you may do.

Chapter two

2 WHAT IS A MODEL, REALLY?

A very large file of numbers. Its size decides almost everything that follows.

Facts in this chapter last checked on 19 October 2026.



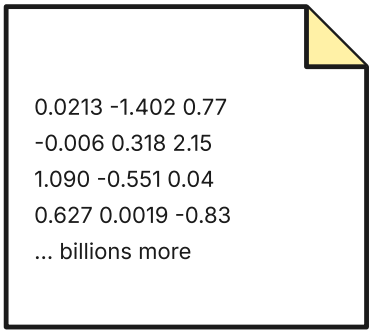
So when we "download a model", what are we actually downloading?

On disk, a model is just a set of very large files full of numbers. Those numbers are called **weights**, and each single number is called a **parameter**.

Learning sets the numbers, answering reads them

When a company **trains** a model, it shows the model huge amounts of text and slowly adjusts the numbers until the model gets good at predicting the next piece of text. This takes thousands of GPUs and weeks of time. You don't need to do it.

When you **run** a model to answer questions, the numbers don't change. The model only reads them. Running a model is called **inference**. Everything in this notebook is about inference.



the weights file



All of it has to fit in GPU memory, with room to spare.

How big is a model?

The size of a model is mostly the number of parameters, multiplied by how many bytes each number takes. "8B" in a model name means about 8 billion parameters.

How many bytes each number takes depends on its **precision**: how many digits it keeps.

Precision	Bytes per parameter	8B model	70B model
FP32 (32-bit)	4	about 32 GB	about 280 GB
BF16 or FP16 (16-bit)	2	about 16 GB	about 140 GB
FP8 or INT8 (8-bit)	1	about 8 GB	about 70 GB
INT4 (4-bit)	about 0.5, plus a little extra	a bit over 4 GB	a bit over 35 GB

Most models are published in **BF16**, so "parameters × 2" is a good first guess. For example, the model we deploy in Chapter 8, Qwen3-4B-Instruct-2507, has about 4 billion parameters stored in BF16. Its weights are about 8 GB.

Sources: Hugging Face's guide to model memory ("roughly 2 × X GB" for X billion parameters in bfloat16), and vLLM's FP8 guide ("a 2x reduction in model memory requirements"). Checked 19 October 2026.

> ⚠ **CAREFUL!**
>
> **The weights are only part of the memory you need.** When the model answers questions, it also needs memory for each conversation it is working on. If the weights fill the whole GPU, there is no room left to answer anyone. Chapters 3 and 4 explain this.

What you actually download

Most open-weight models are shared on **Hugging Face**, a website that hosts models. A model's page usually has:

- **The weight files**, often ending in `.safetensors`. A big model is split into many files.
- `config.json`, which describes the model's shape: how many layers it has, how long a conversation it can handle, and so on.
- **The tokenizer files**, which turn text into tokens and back.
- **The model card**, the page you read first. It tells you what the model is for, how long a conversation it supports, how to run it, and its license.

Some models are **gated**. You must accept the license on the website first, then use an access token when you download. Chapter 8 shows where that token goes.

Bigger is not always better

Bigger models usually give better answers, but they need more GPU memory, cost more to run and answer more slowly. A smaller model that is good at your task can be the better choice. The only way to know is to test both on your own tasks.

> </> **GEEK CORNER** *(skip it if code isn't your thing)*
>
> **Mixture of experts (MoE).** Some models, such as Qwen3-235B-A22B or gpt-oss-120b, are split into many "experts". For each token, only a few experts do the work. In Qwen3-235B-A22B, about 22 billion of the 235 billion parameters are used for each token, which makes it faster than a normal model of that size. **But all 235 billion parameters still have to be in GPU memory**, because the next token may need different experts. Plan memory for the full size.

Read the license before you build

Open-weight models come with different licenses. Here are the main families as of October 2026:

Model family	License
Qwen3 (Alibaba)	Apache 2.0
gpt-oss (OpenAI)	Apache 2.0
Gemma 4 (Google)	Apache 2.0. Gemma 3 and older use Google's Gemma Terms of Use.
Mistral Large 3 and Ministral 3 (Mistral AI)	Apache 2.0
DeepSeek V3.2	MIT
Llama 4 (Meta)	Llama 4 Community License, Meta's own license with its own conditions

Apache 2.0 and **MIT** are well-known licenses that allow commercial use. A model maker's own license can add conditions, for example about how many users you have, how you name things you build, or what you may use the model for. Read the actual license text on the model card, and ask your legal team if it is unclear.

Careful!

Licenses change between versions of the same family. Gemma 4 moved to Apache 2.0, but Gemma 3 did not. Always check the license of the exact model and version you use, not the family name.

There are no silly questions

Q: Does the model learn from our users while it runs?

A: No. During inference the weights don't change. The model only "remembers" what is in the current conversation. Learning from your data would need a separate training step called fine-tuning.

Q: What does BF16 mean?

A: It is a way to store each number in 16 bits (2 bytes) instead of 32. It keeps enough detail for models to work well and halves the memory. "B" stands for "brain", because Google Brain designed it.

Q: Is a 4-bit model just a worse model?

A: It is the same model with less detailed numbers. Often the answers are nearly as good, sometimes noticeably worse. Chapter 4 covers this trade-off, called quantization.

Your turn

Work out the size of the weights. Use 2 bytes for BF16, 1 byte for FP8 and about 0.5 bytes for INT4.

1. A 14B model in BF16.

2. A 70B model in FP8.

3. An 8B model in INT4.

4. A mixture-of-experts model with 30 billion parameters in total, of which 3 billion are used for each token, stored in BF16.

The answers are at the back.

Key takeaways

- A model is a set of files full of numbers called weights. Each number is a parameter.
- Running a model to answer questions is called inference. The weights don't change.
- Size in GB is roughly parameters × bytes per parameter. BF16 is 2 bytes, FP8 is 1, INT4 is about 0.5.
- The weights are not the only thing in GPU memory. Each conversation needs room too.
- Mixture-of-experts models use part of their weights per token, but all of them must be loaded.
- Check the license of the exact model version before you build on it.

Chapter three

3 How a model writes an answer

Tokens, the two stages of every answer, and the notes the model keeps on the side.

Facts in this chapter last checked on 19 October 2026.



Why does my answer start slowly and then flow word by word?

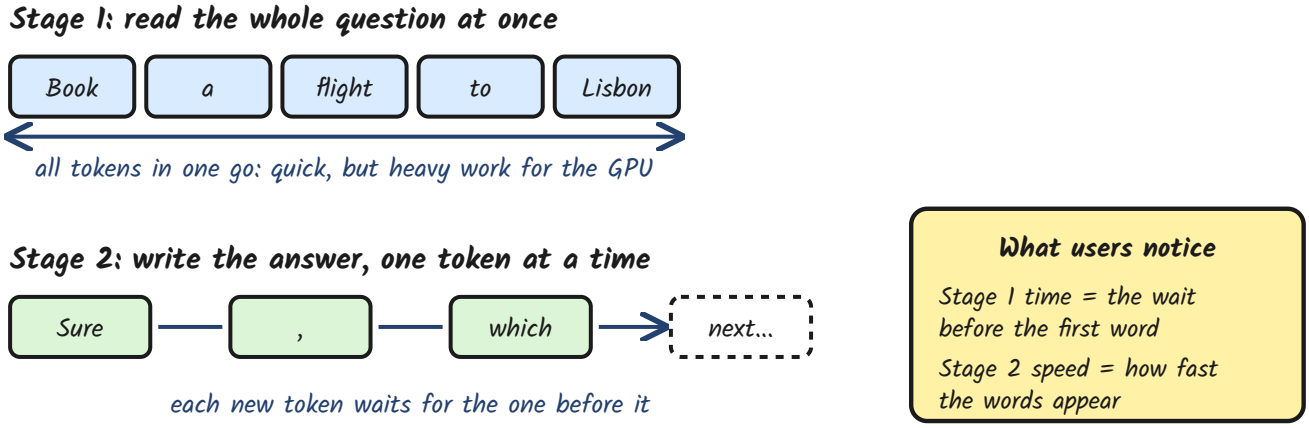
To size a GPU, or to understand a bill, you need to know what the model does when a request arrives.

Everything is tokens

A model does not read letters or whole sentences. It reads **tokens**. A token is a small piece of text: often a short word, part of a longer word, or a punctuation mark. A tool called the **tokenizer** cuts text into tokens on the way in, and joins tokens back into text on the way out.

Everything is counted in tokens: the length of a conversation, the speed of the model, and the price on a model service.

Two stages in every answer



Stage 1: read the question. The model takes in all the tokens of the request at once: the system instructions, the conversation so far, the tool results, and the new question. This stage is called **prefill**. The GPU does a lot of work, but it does it all in one go.

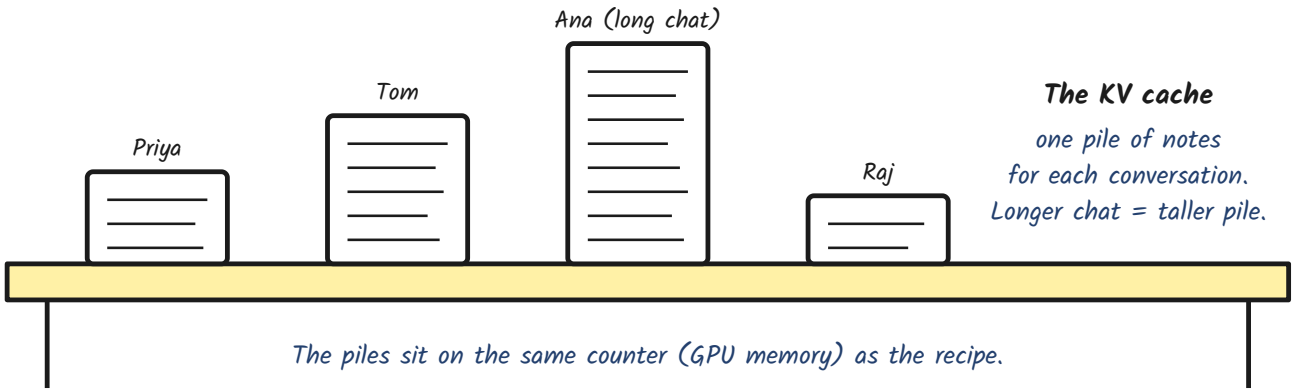
Stage 2: write the answer. The model writes the answer one token at a time. Each new token depends on all the tokens before it, so the model can't write token 10 before it has written token 9. This stage is called **decode**. It is the reason answers appear word by word.

Think about it

Botley's requests are long: instructions, tool results and the chat history, often thousands of tokens. Its answers are often short. Which stage do you think takes most of Botley's GPU time?

The notes on the counter: the KV cache

While it writes, the model needs to look back at every token in the conversation. Working everything out again for each new token would be very slow. So the model keeps notes about every token it has already seen. These notes are called the **KV cache** ("key-value cache").



The KV cache lives in **GPU memory**, next to the weights. It has two important properties:

- **It grows with the length of the conversation.** A long chat with many tool results makes a tall pile of notes.
- **Every conversation the GPU works on has its own pile.** Twenty conversations at once means twenty piles.

So the free GPU memory after the weights decides how many conversations the model can handle at the same time, and how long they can be.

</> Geek corner *(skip it if code isn't your thing)*

How big is one pile of notes? For each token, the model stores 2 × (number of layers) × (number of KV heads) × (head size) numbers. The model from Chapter 8, Qwen3-4B-Instruct-2507, has 36 layers, 8 KV heads and a head size of 128, all in BF16 (2 bytes):

```
2 × 36 × 8 × 128 × 2 bytes = 147,456 bytes ≈ 147 KB per token
```

One conversation of 8,192 tokens needs about 1.2 GB of notes. Ten of them at once need about 12 GB, more than the model's own 8 GB of weights. You can find these numbers in the model's `config.json`.

Cooking many orders at once

A GPU is very good at doing the same calculation on lots of data at the same time. Writing tokens for only one conversation leaves most of it idle. So serving software puts many conversations together and writes the next token for all of them in one step. This is called **batching**.

Here is a real example. NVIDIA tested an 8B model (Llama 3.1 8B in FP8) on one L40S GPU, with 1,000 tokens in and 1,000 tokens out per request:

Users at the same time	Tokens written per second, all users together	What each user sees
1	about 71	about 71 tokens per second
100	about 2,869	about 29 tokens per second, and about 0.8 seconds before the first token

Source: NVIDIA NIM LLM benchmarks (NIM 1.8.0). These numbers come from NVIDIA's own serving software, not vLLM, and they depend heavily on the request length and settings. Use them to understand the shape, not as a promise. Checked 19 October 2026.

With 100 users, the GPU writes about 40 times more tokens per second in total. Each user's answer flows more slowly, but still faster than people read. This is why one GPU can serve many users, and why a GPU that serves only one user at a time is a very expensive way to run a model.

Three speed numbers that matter

Number	What it means	Who cares
Time to first token	How long the user waits before the answer starts. Mostly stage 1.	Users of a chat agent.
Time between tokens	How fast the answer flows once it starts. Stage 2.	Users reading a long answer.
Throughput	Tokens per second for all users together.	Whoever pays for the GPU.

These pull against each other. More users at once means better throughput, but slower answers for each user. Running your own model means you choose the balance. On a model service, the provider chooses.

There are no silly questions

Q: Is one token the same as one word?

A: Not exactly. Common short words are often one token. Long or unusual words are split into several. Different models cut text in different ways, so the same text can be a different number of tokens on different models.

Q: Why are output tokens usually more expensive than input tokens?

A: Input tokens are read together in stage 1. Output tokens are written one at a time in stage 2, and each one takes a full step through the model. On many model services, output tokens cost more for this reason.

Q: Does a longer context window cost more?

A: Only if you use it. A model may support 262,144 tokens, but every token in a conversation needs space in the KV cache. Long conversations use more GPU memory and take longer in stage 1.

Your turn

1. Botley's answers start after a long pause, then flow quickly. Which stage is slow?

2. Botley's answers start quickly but then appear very slowly. Which stage is slow?

3. Using the Geek corner numbers, how much KV cache do 20 conversations of 4,096 tokens each need?

4. Why does a GPU that serves one user at a time cost so much per token?

The answers are at the back.

Key takeaways

- Models read and write tokens, small pieces of text.
- Every answer has two stages: reading the whole request at once (prefill), then writing one token at a time (decode).
- The KV cache holds notes for every token in every active conversation. It lives in GPU memory and grows with conversation length.
- Batching many conversations together makes the GPU far more efficient.
- Watch three numbers: time to first token, time between tokens, and throughput.



Chapter four

Meet the GPU

GPU memory is the counter space in your kitchen. Everything has to fit on it.

Facts in this chapter last checked on 19 October 2026.



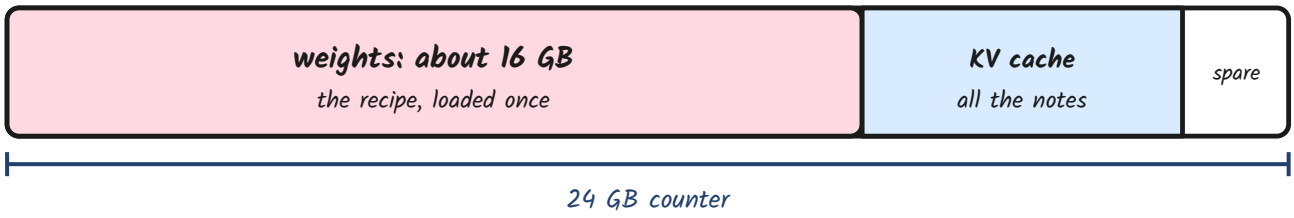
There are so many GPUs. How do I know which one is big enough?

A normal computer chip (a CPU) is like a few very skilled chefs. A **GPU** is like thousands of kitchen helpers who each do one simple job, all at the same time. Running a model takes billions of simple multiplications for every token, so a GPU does it far faster than a CPU.

Memory is the first question

When you pick a GPU for a model, the first question is not speed. It is **memory**. The weights must be in the GPU's own memory, and the KV cache from Chapter 3 needs room next to them.

One GPU with 24 GB of memory, running an 8B model in BF16



Whatever is left after the weights decides how many conversations fit at once, and how long they can be.

Think of GPU memory as counter space. The recipe (the weights) takes a fixed part. The notes for every order (the KV cache) take the rest. If the recipe fills the whole counter, you can't cook anything.

Serving software such as vLLM takes a share of GPU memory when it starts. By default, vLLM uses 92% of it. It loads the weights, keeps some room for its own work, and gives everything else to the KV cache.

The GPU line-up

These are NVIDIA GPUs you can rent on the three big clouds in October 2026, smallest first:

GPU	Memory per GPU	AWS	Google Cloud	Azure
L4	24 GB	g6	G2	—
A10G (A10 on Azure)	24 GB	g5	—	NVads A10 v5 (made for graphics, shares the GPU)
L40S	48 GB	g6e	—	—
A100	40 GB or 80 GB	p4d (40 GB), p4de (80 GB)	A2 (40 GB), A2 Ultra (80 GB)	NC A100 v4 (80 GB), ND A100 v4
RTX PRO 6000	96 GB	g7e	G4	NC RTX PRO 6000 BSE v6
H100	80 GB (94 GB in the NVL version)	p5	A3	ND H100 v5, NC H100 v5 (NVL)

GPU	Memory per GPU	AWS	Google Cloud	Azure
H200	141 GB	p5e, p5en	A3 Ultra	ND H200 v5
B200	about 180 GB	p6-b200	A4	—

Sources: NVIDIA product pages, AWS EC2 accelerated computing instance types, Google Cloud GPU machine types, and Azure GPU VM sizes. A dash means we found no matching size on that cloud. Clouds add new GPUs often, so check before you plan. Checked 19 October 2026.

Many machine types come in sizes with 1, 2, 4 or 8 GPUs. The big ones (H100 and up) are mostly sold in machines with 8 GPUs. AWS also has a single-H100 size, p5.4xlarge. On Google Cloud, the 1, 2 and 4 GPU sizes of A3 High are only available as Spot or Flex-start machines, not normal on-demand ones.

What they cost

Here are some on-demand Linux prices, per hour, for the whole machine:

Machine	GPUs	Price per hour
AWS g6.xlarge (us-east-1)	1 × L4	$0.80
Google g2-standard-8 (us-central1)	1 × L4	$0.85
AWS g6e.xlarge (us-east-1)	1 × L40S	$1.86
Azure NC24ads A100 v4 (East US)	1 × A100 80 GB	$3.67
AWS p5.4xlarge (us-east-1)	1 × H100	$6.88
Azure NC40ads H100 v5 (East US)	1 × H100 NVL	$6.98
AWS p5.48xlarge (us-east-1)	8 × H100	$55.04

Sources: AWS EC2 on-demand pricing, Google Cloud accelerator-optimized VM pricing and Azure Linux VM pricing, checked 19 October 2026. Prices change and differ by region. Spot machines can cost much less (all three clouds advertise up to about 90% off), but the cloud can take them back at short notice. Committing for one or three years also lowers the price.

A month has about 730 hours. So one L4 machine running all month costs about $590 on AWS, and one 8 × H100 machine about $40,000.

Will it fit?

A simple way to check:

1. Work out the weights: parameters × bytes per parameter (Chapter 2).
2. Take about 90% of the GPU memory. That is roughly what the serving software can use.
3. Subtract the weights. What is left is for the KV cache. If it is nothing, or very little, the model does not fit in practice.

For example: an 8B model in BF16 on an L4. The weights are about 16 GB. 90% of 24 GB is about 21.6 GB. That leaves about 5.6 GB for the KV cache. It fits, but with room for only a modest number of conversations at once.

Making the model smaller: quantization

Quantization means storing the weights with fewer bits: 8 or 4 instead of 16. The model takes less memory and often runs faster.

- **FP8** halves the memory compared with BF16. Newer GPUs such as the L4, L40S and H100 have built-in support for FP8 math. Older ones such as the A100 and A10G do not, so FP8 helps them less.
- **INT4** takes about a quarter of the memory of BF16. More of the model's quality can be lost.

Many model makers publish ready-made FP8 or 4-bit versions of their models. Use those when you can, and test the answers on your own tasks.

Careful!

Quantization can quietly change answers. A 4-bit model may be fine for chat but worse at following strict tool-call formats. Botley books real flights. Test the quantized model on Botley's real tasks before it goes live.

When one GPU is not enough

A 70B model in BF16 needs about 140 GB for its weights alone. Only the B200 can hold that on one GPU, and it is rare and costly. The usual answer is to split the model across several GPUs in the same machine. This is called **tensor parallelism**: each GPU holds part of every layer, and they work on each token together. In vLLM, you set it with `--tensor-parallel-size`.

For example, NVIDIA's own tests run Llama 3.3 70B in FP8 (about 70 GB of weights) on 2 or 4 H100 GPUs.

Not only NVIDIA

- **Google TPUs** are Google's own chips: TPU v5e (16 GB per chip), v6e "Trillium" (32 GB) and TPU7x "Ironwood" (192 GiB), which became generally available on 31 March 2026.
- **AWS Inferentia2** (32 GB per chip) and **Trainium** are Amazon's own chips.

They can be cheaper for some models, but they need software built for them. Check that your model and serving tool support them before you choose them.

Careful!

New cloud accounts usually can't start GPU machines at all. On AWS, the default quota for G and P instances (the GPU families) is 0. On Google Cloud, free trial accounts can't use GPUs, and you need GPU quota for your region. On Azure, quota is set per machine family. Request more quota early. It can take days, and popular GPUs can be hard to get in some regions.

There are no silly questions

Q: Should I just get the biggest GPU?

A: No. A big GPU running a small model is mostly idle, and you pay for every hour. Pick the smallest GPU that fits your model with enough room for the KV cache, then measure.

Q: Can I use Spot machines for a live agent?

A: Only with care. The cloud can take a Spot machine back with very little warning, and getting a new GPU can take minutes. Spot works better for batch jobs, or as extra capacity next to normal machines.

Q: Why is GPU memory such a big deal? My laptop has 32 GB of RAM.

A: The model must be in the GPU's own memory, which is separate from the computer's normal RAM. GPU memory is much faster, and much more expensive.

Your turn

Will it fit? Use "weights must be well under 90% of GPU memory".

1. An 8B model in BF16 on one L4 (24 GB).

2. A 70B model in BF16 on one H100 (80 GB).

3. A 70B model in FP8 on one H200 (141 GB).

4. A 4B model in BF16 on one L4 (24 GB). How much is left for the KV cache?

The answers are at the back.

Key takeaways

- GPU memory is the first thing to check. Weights and KV cache must both fit.
- vLLM uses 92% of GPU memory by default and gives what is left after the weights to the KV cache.
- GPUs range from 24 GB (L4, A10G) to about 180 GB (B200). Prices range from under $1 to over $100 per hour per machine.
- Quantization (FP8, INT4) shrinks models. Test the quality before you rely on it.
- Big models are split across several GPUs with tensor parallelism.
- Ask for GPU quota early. New accounts often start with none.



Chapter five

The managed path

Amazon Bedrock, Google's Agent Platform and Microsoft Foundry: what they offer, how they charge, and what they promise about your data.

Facts in this chapter last checked on 19 October 2026.



Before we buy any GPUs, what can the cloud do for us?

Each big cloud has a model service. You send requests, the cloud runs the model on its GPUs, and you never see a GPU. Each one offers the same four options. Only the names are different.

What you want	AWS: Amazon Bedrock	Google: Agent Platform	Microsoft Foundry
Shared, pay per token	On-demand (Standard, Flex and Priority tiers)	Standard, Priority and Flex PayGo	Serverless: Global Standard, Data Zone Standard, Standard
Cheaper, not right away	Batch inference	Batch inference	Global Batch, Data Zone Batch
Reserved capacity	Provisioned Throughput, Reserved tier	Provisioned Throughput	Provisioned (PTU)
An open model on GPUs only you use	Custom Model Import, Bedrock Marketplace	Self-deployed open models (Model Garden)	Managed compute



CAREFUL!

Two services changed their names in 2026. Google renamed Vertex AI to **Gemini Enterprise Agent Platform** ("Agent Platform" for short) on 22 April 2026. Microsoft renamed Azure AI Foundry to **Microsoft Foundry**. Many web pages, and even some API addresses, still use the old names.

Shared, pay per token: the restaurant

This is how Botley works today. You pay a price per million tokens in, and another per million tokens out. You pay nothing when nobody uses it, and you don't wait for any machine to start.

Here are prices for two popular open-weight models on all three clouds, so you can compare like with like:

Model	Amazon Bedrock	Google Agent Platform	Microsoft Foundry (Global)
Llama 3.3 70B	$0.72 in / $0.72 out	$0.72 in / $0.72 out	$0.71 in / $0.71 out
gpt-oss-120b	$0.15 in / $0.60 out	$0.09 in / $0.36 out	$0.15 in / $0.60 out

Prices in US dollars per million tokens, on-demand, US regions (Bedrock us-east-1, Foundry East US). Sources: AWS Price List for Bedrock, Google Cloud generative AI pricing, Azure retail prices. Checked 19 October 2026.

The tiers trade price for speed. On Bedrock, **Flex** is 50% cheaper but slower, and **Priority** costs 75% more and is served first. Google's Agent Platform has Flex and Priority PayGo with the same idea, and Microsoft Foundry offers Flex and Priority processing on Global Standard.

Microsoft's names tell you **where your requests are processed**. **Global Standard** can use Microsoft's data centers anywhere. **Data Zone Standard** stays inside a zone: the US, the EU or Asia Pacific. **Standard** keeps it in the Azure geography where you created the resource. AWS does something similar with **cross-region inference**: a

"geographic" profile keeps requests in the US, the EU or Asia Pacific, and a "global" one can use any region and is about 10% cheaper.

Batch: cheaper, but not right away

You send a big file of requests and collect the answers later. All three clouds charge 50% less for batch on the models that support it. Microsoft's target is to finish within 24 hours. Batch is no good for a live chat with Botley, but very good for jobs like "summarize yesterday's 40,000 support tickets".

Reserved capacity: the private room

You pay by the hour for a fixed amount of model capacity, whether you use it or not. In return, your requests don't compete with other customers for that capacity.

- **Bedrock Provisioned Throughput:** billed per hour for each "model unit". You can buy it with no commitment, or commit for 1 or 6 months. Some models you fine-tune on Bedrock can only be run this way. Bedrock also has a **Reserved tier**: you reserve tokens per minute for 1 or 3 months, and traffic above that is served at Standard prices.
- **Agent Platform Provisioned Throughput:** billed in units called GSUs, for a fixed term of 1 week (Google's own models only), 1 month, 3 months or 1 year. Traffic above what you bought is charged at normal pay-per-token prices.
- **Foundry Provisioned (PTU):** billed per hour for each Provisioned Throughput Unit, or at a lower price if you reserve PTUs for 1 month or 1 year.

A "model unit", a GSU and a PTU are measures of model capacity. They are not GPUs. Each provider has a calculator to work out how many you need for your traffic.

An open model on GPUs only you use

This is the middle path from Chapter 1. You pick an open-weight model, the provider runs it on GPUs that only you use, and you don't manage the servers.

- **Bedrock Custom Model Import:** you upload the weights of a supported model family (including Llama, Mistral, Mixtral, Qwen and gpt-oss) and call it like any Bedrock model. Importing is free. You pay per minute for each copy of the model that is running, counted in 5-minute blocks, plus a monthly storage fee.
- **Bedrock Marketplace:** over 100 more models, deployed to SageMaker AI endpoints and called through Bedrock.
- **Agent Platform self-deployed open models:** you choose a model from Model Garden and the machine and GPU to run it on. Google says these "aren't serverless". You pay per node per hour, including when it is idle.
- **Foundry managed compute:** the model runs on virtual machines that only you use, billed per hour for the machines.

This gives you the model you choose and steady speed, without running Kubernetes. But like your own GPUs, you pay by the hour, even when nobody is using it.

What they promise about your data

Sam's second question was about data. Here is what each provider says, in its own words:

Provider	What it says
Amazon Bedrock	Your content "is not used to improve the base models and is not shared with any model providers."
Google Agent Platform	Google "won't use your data to train or fine-tune any AI/ML models without your prior permission or instruction."
Microsoft Foundry	Prompts and answers "are NOT available to OpenAI or other providers" and "are NOT used to train any generative AI foundation models without your permission."

⚠ Careful!

"Not used for training" is not the same as "never stored". Each provider can keep some data for a while, for example for abuse monitoring or caching. Google's Gemini caches data by default for up to 24 hours, and you can turn this off. Microsoft may keep flagged requests for human review unless you are approved to switch that off. Bedrock has a data-retention setting for each region. Read the data pages for the exact service and model you use, and set them before you go live.

Models don't last forever

On a model service, the provider decides when a model version is switched off.

- **Bedrock:** a model goes from Active to Legacy to End-of-Life. Most get at least 6 months' notice, some only 45 days. After End-of-Life, requests fail.
- **Microsoft Foundry:** generally available models usually retire 18 months after launch (12 months for models from some partners such as Anthropic, DeepSeek and Mistral). You get at least 60 days' notice, and the dates can't be extended.

So on the managed path, plan to test and move to new model versions a few times a year.

Pay per token vs reserved capacity

Pay per token	Reserved capacity
You pay nothing when nobody uses Botley. Nights and weekends are free.	But at busy times I share the kitchen with every other customer. I want my own room.
You pay for your room every hour, even at 3 a.m. when it's empty.	If Botley is busy all day and night, that hourly price can work out cheaper per token.
Only if you really are that busy. Most new agents aren't.	

There are no silly questions

Q: Can I use Claude through the cloud services?

A: Yes. Claude is available on Bedrock, on Google's Agent Platform and on Microsoft Foundry, as a managed model. You still can't download it.

Q: Does a managed service keep my data in my country?

A: It can, if you choose the right options. To stay in one country, pick a region in that country, use a Standard deployment on Foundry (it stays in that Azure geography), and call Bedrock in that region without a cross-region profile. Data zones and geographic profiles cover groups of countries, such as the EU. Check the documents for the exact model, because not every model is offered in every region.

Q: Which managed option is cheapest?

A: It depends on how busy you are. Pay per token is cheapest for low or uneven traffic. Reserved capacity and dedicated GPUs only win when you keep them busy most of the time. Chapter 9 shows how to work it out.

Your turn

Which option would you choose: **pay per token**, **batch**, **reserved capacity**, or **an open model on GPUs only you use**?

1. Every night, Botley's team wants to summarize 50,000 old booking emails. Nobody needs the results before morning.

2. Botley gets a few hundred chats a day, mostly during office hours.

3. A partner wants to use a fine-tuned open model, but the team doesn't want to run Kubernetes.

4. A busy airline helper gets steady traffic 24 hours a day and needs the same speed even when other customers are busy.

The answers are at the back.

Key takeaways

- Bedrock, Agent Platform and Foundry each offer pay per token, batch, reserved capacity, and dedicated GPUs for open models.
- Vertex AI is now Gemini Enterprise Agent Platform. Azure AI Foundry is now Microsoft Foundry.
- Batch costs 50% less but is not for live chat.
- Reserved capacity and dedicated GPUs are billed by the hour, busy or not.
- All three promise not to train on your data without permission, but some data can still be stored for a while. Check the settings.
- Managed models are retired on the provider's schedule. Plan for regular upgrades.

Chapter six

6 THE DO-IT-YOURSELF PATH

vLLM: the head chef that turns a GPU and a model into a service Botley can call.

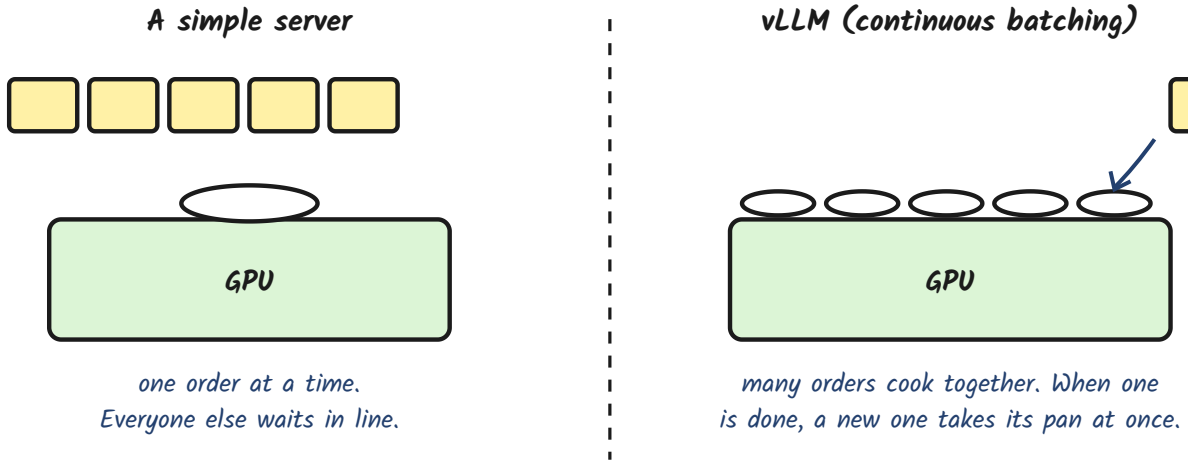
Facts in this chapter last checked on 19 October 2026.



A GPU and a weights file. How do I turn that into something Botley can call?

A GPU and a weights file are not a service yet. You need a program that loads the weights into the GPU, listens for requests, puts conversations into batches, looks after the KV cache, and sends answers back. This program is called an **inference server** or **serving engine**. One of the most widely used open-source ones is **vLLM**.

What vLLM does well



Continuous batching. A simple server takes a group of requests, finishes all of them, then takes the next group. Short answers wait for long ones. vLLM adds a new request to the running batch as soon as there is room, at every step. The GPU stays busy and nobody waits for a whole group to finish.

PagedAttention. The KV cache is stored in small fixed-size blocks, like pages in a notebook, instead of one big reserved area per conversation. A conversation gets a new block only when it needs one. vLLM's authors say this cuts wasted KV cache memory to under 4%. Less waste means more conversations fit on the same GPU.

It speaks the OpenAI API. vLLM answers requests in the same format as OpenAI's API, at addresses such as `/v1/chat/completions` and `/v1/models`. Most agent frameworks can already talk to that format.

Botley barely changes

Because vLLM speaks the OpenAI format, Maya only changes two settings in Botley: the address of the model and the model's name.

</> Geek corner *(skip it if code isn't your thing)*

```python
from openai import OpenAI

# Before: a model service. After: your own vLLM server.
client = OpenAI(
    base_url="http://vllm.models.svc.cluster.local/v1",
    api_key="the-key-you-gave-vllm",
)

reply = client.chat.completions.create(
    model="Qwen/Qwen3-4B-Instruct-2507",
    messages=[
        {"role": "user", "content": "Find me a flight to Lisbon on Tuesday."},
    ],
)
print(reply.choices[0].message.content)
```

The model name must match the name vLLM uses, which is the model you started it with unless you set `--served-model-name`.

⚠ Careful!

Same format does not mean same behavior. The request format is the same, but the model is different. Tool calls, long instructions and answer style can all change. Also, some features need extra vLLM settings. For example, tool calling needs flags that depend on the model, so check the vLLM docs for your model. Test Botley's real tasks before you switch.

Try it on one machine

The quickest way to see vLLM work is its official Docker image on a single machine with an NVIDIA GPU. This is vLLM's own example, using a tiny model:

#	Step	What you type	What happens
1	Start vLLM	`docker run --runtime nvidia --gpus all \ -v ~/.cache/huggingface:/root/.cache/huggingface \ --env "HF_TOKEN=$HF_TOKEN" \ -p 8000:8000 \ --ipc=host \ vllm/vllm-openai:latest \ --model Qwen/Qwen3-0.6B`	Downloads the image and the model, loads the model into the GPU and listens on port 8000. The `-v` line keeps downloaded models on your disk, so the next start is faster. `--ipc=host` gives vLLM the shared memory it needs.
2	Is it ready?	`curl http://localhost:8000/health`	Answers with success (HTTP 200) once the model is loaded.
3	Which models?	`curl http://localhost:8000/v1/models`	Lists the model name that vLLM is serving.
4	Ask something	`curl http://localhost:8000/v1/chat/completions \ -H "Content-Type: application/json" \ -d '{"model": "Qwen/Qwen3-0.6B", "messages": [{"role": "user", "content": "Say hello"}]}'`	Sends one chat message in the OpenAI format and returns the answer.

The `HF_TOKEN` line is only needed for gated models (Chapter 2). The image is large, about 9 GB compressed. Source: vLLM's Docker deployment guide, checked 19 October 2026.

The settings you will actually use

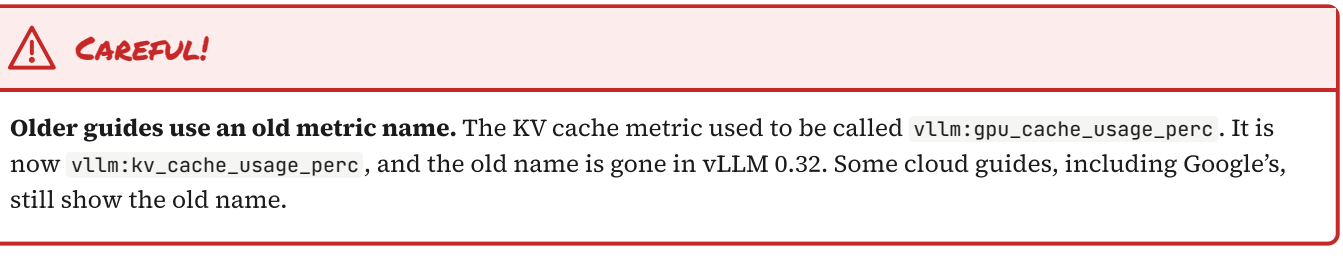
Setting	What it does	Default
`--model` (or the first word after `vllm serve`)	Which model to load, by its Hugging Face name or a local folder.	none
`--max-model-len`	The longest conversation, in tokens, that vLLM will accept. Set it lower to save KV cache memory.	the model's own maximum
`--gpu-memory-utilization`	How much of the GPU's memory vLLM may use, from 0 to 1.	0.92
`--tensor-parallel-size`	How many GPUs to split the model across (Chapter 4).	1
`--quantization`	Which quantization method to use, if the model needs one set.	worked out from the model
`--dtype`	The number format for the weights, such as `bfloat16`.	`auto`
`--served-model-name`	The name Botley uses in its requests.	the `--model` value
`--api-key`	A key that requests to `/v1` addresses must send.	none
`--port`	The port vLLM listens on.	8000

Source: vLLM `serve` command reference, version 0.32.0 (released 18 October 2026). vLLM releases a new version about every two weeks, so check the docs for the version you use.

Built-in health and numbers

vLLM gives you two more addresses for free:

- `/health` answers when the server is ready. Kubernetes uses it to know when to send traffic (Chapter 7).
- `/metrics` gives numbers in Prometheus format. The most useful are `vllm:num_requests_running` (conversations being worked on now), `vllm:num_requests_waiting` (conversations waiting for room), `vllm:kv_cache_usage_perc` (how full the KV cache is, from 0 to 1) and `vllm:time_to_first_token_seconds`.





Other serving engines

Tool	What it is	When to look at it
SGLang	Another open-source serving engine, built for large-scale serving and agent workloads.	You want to compare speed with vLLM on your own model.

Tool	What it is	When to look at it
TensorRT-LLM	NVIDIA's open-source library for fast inference on NVIDIA GPUs.	You want maximum speed on NVIDIA hardware and can spend time tuning.
NVIDIA NIM	NVIDIA's ready-made model containers. NIM for LLMs version 2.0 is built on vLLM.	You want NVIDIA's packaged and supported version.
Hugging Face TGI	Hugging Face's serving engine, now in maintenance mode. Hugging Face recommends vLLM or SGLang instead.	Only if you already run it.
Ollama and llama.cpp	Easy ways to run models on your own computer. Ollama only listens on your own computer and handles one request at a time by default.	Trying models on a laptop, not serving many users.

There are no silly questions

Q: Is vLLM free?

A: Yes, vLLM is open source. You pay for the GPUs it runs on and for your team's time.

Q: Should I use the "latest" image in production?

A: No. `latest` changes every couple of weeks. Use a fixed version tag, such as the version you tested, and upgrade on purpose.

Q: Can vLLM serve more than one model?

A: One vLLM server serves one base model. It can also serve small add-ons for that model, called LoRA adapters. To serve two different models, run two servers. Each one needs its own GPU memory.

Your turn

1. Botley gets errors because its requests are longer than vLLM allows. Which setting do you look at?

2. Requests are queuing up and users wait a long time. Which metric shows this?

3. A teammate wants to put Ollama on a cloud GPU and point all of Botley's users at it. What do you tell them?

4. Why can Maya switch Botley to vLLM by changing only two settings?

The answers are at the back.

Key takeaways

- A serving engine turns a GPU and a weights file into a service. vLLM is one of the most widely used open-source ones.
- Continuous batching and PagedAttention let vLLM serve many conversations on one GPU with little waste.
- vLLM speaks the OpenAI API format, so Botley only changes the address and the model name.
- Watch `/health` and `/metrics`, especially requests waiting and KV cache usage.
- vLLM has no real security of its own. Keep it on a private network.
- Pin a fixed vLLM version. It changes every couple of weeks.



Chapter seven

Models on Kubernetes

GPU nodes, slow starts, the right way to scale, and why you always run two.

Facts in this chapter last checked on 19 October 2026.



It took my model six minutes to wake up. My old app took six seconds!

One vLLM container on one machine is fine for a test. For a live service, you want something that restarts it when it crashes, runs more than one copy, and adds copies when it gets busy. For most teams that is **Kubernetes**: Amazon EKS, Google GKE or Azure AKS (Notebook #1, Chapter 5).

Running a model on Kubernetes works like running any other container, with four big differences.

Difference 1: the machines need GPUs, and Kubernetes must see them

You add a **GPU node pool**: a group of machines that have GPUs. Then a pod asks for a GPU in its settings:

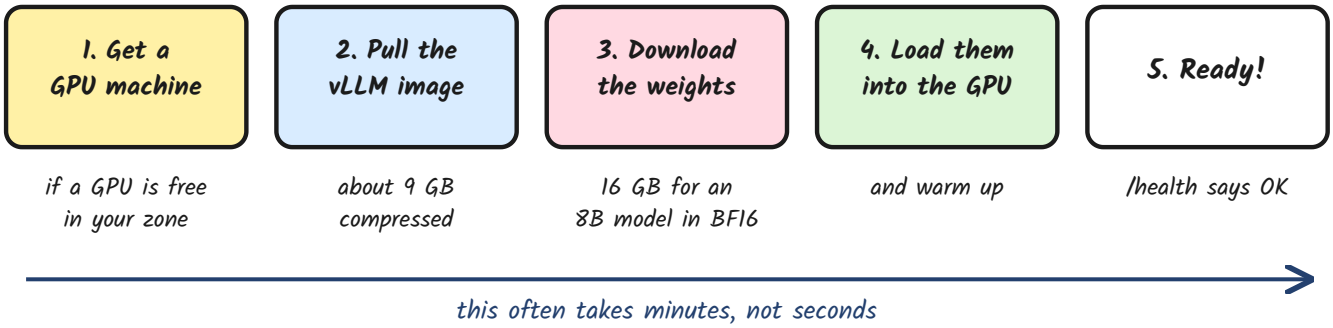
```
resources:
  limits:
    nvidia.com/gpu: 1
```

Kubernetes then places the pod only on a machine with a free GPU. For this to work, each GPU machine needs two things: the **NVIDIA driver**, and the **NVIDIA device plugin**, which tells Kubernetes how many GPUs the machine has. Each cloud handles this a little differently:

Cloud	Driver	Device plugin
Google GKE	Installed for you by default (GKE 1.30.1-gke.1156000 and later). Autopilot clusters handle it completely.	Handled by GKE.
Amazon EKS	Included in the EKS-optimized NVIDIA machine images.	**Not** included in the Amazon Linux 2023 NVIDIA images, so you install it. It is included in the Bottlerocket NVIDIA images. EKS Auto Mode handles both.
Azure AKS	Installed by default.	You install it. A "managed GPU node pool" option that also installs it is in preview. The KAITO add-on (Chapter 8) sets everything up for you.

Sources: GKE "About GPUs", Amazon EKS optimized accelerated AMIs and EKS Auto Mode docs, Azure AKS "Use GPUs" and AKS-managed GPU node pools docs. Checked 19 October 2026. NVIDIA's **GPU Operator** can install all of these parts for you on any cluster.

Difference 2: starting takes minutes, not seconds



A normal app container starts in seconds. A model server has much more to do: the cluster may need to start a new GPU machine, the vLLM image is about 9 GB, the weights can be many GB more, and then they must be loaded into the GPU. Together this can take several minutes.

This changes how you run it:

- **Keep copies running.** Don't scale a live model down to zero and expect the first user to wait.
- **Keep the weights close.** Downloading from Hugging Face on every start is slow and depends on their service. Store a copy in your own cloud storage or on a disk in the cluster. vLLM can also stream weights straight from Amazon S3 or Google Cloud Storage with a tool called the Run:ai Model Streamer.
- **Make the image arrive faster.** GKE can preload images onto machines with secondary boot disks. AWS can use SOCI (Seekable OCI), which lets a container start before the whole image has downloaded.
- **Tell Kubernetes to wait.** Add a startup probe on vLLM's `/health` that allows several minutes, so Kubernetes doesn't restart the pod while the model loads. Add a readiness probe on `/health` too, so Kubernetes only sends traffic when the model is ready.

> ⚠ **CAREFUL!**
>
> **vLLM needs shared memory.** Containers get very little shared memory (`/dev/shm`) by default, and vLLM can fail without more. On Kubernetes, mount an in-memory volume at `/dev/shm`. vLLM's own Kubernetes guide shows how, and Chapter 8 includes it.

Difference 3: don't scale on CPU

Normal apps scale when CPU use goes up. For a model server, CPU tells you almost nothing, because the work happens on the GPU. Even GPU use is a poor guide. Google's GKE guide says GPU utilization "does not measure how much work is being done".

Scale on what vLLM itself reports instead. Google's best-practice guide for GKE suggests:

Goal	Scale on
Serve as many requests as possible	`vllm:num_requests_waiting`, the queue of conversations waiting for room
Keep answers fast	KV cache usage (`vllm:kv_cache_usage_perc`), or `vllm:num_requests_running`

To use these numbers, you need two layers:

1. **More pods:** Kubernetes' built-in autoscaler (the HorizontalPodAutoscaler), set up to read vLLM's numbers, or **KEDA**, which can scale on a Prometheus query.
2. **More GPU machines** for those pods: the cluster autoscaler, **Karpenter** on EKS, node auto-provisioning on AKS, or GKE's own autoscaling and Autopilot.

Think about it

Adding a copy of the model takes minutes (Difference 2). What does that mean for when your autoscaler should act? Should it wait until users are already waiting a long time?

Difference 4: always run at least two

One copy of the model means one point of failure. If its machine is replaced, updated or simply fails, Botley loses its brain until a new GPU machine starts and loads the model, which can take minutes. Run at least two copies, on two different machines. That also lets you update one at a time.

Yes, that doubles the GPU bill. Chapter 9 includes this in the math.

Smarter routing (you don't need this on day one)

A normal load balancer sends requests to pods in turn. For model servers, it helps to send each request to the pod with the most free KV cache, or to the pod that has recently seen the same long instructions. The Kubernetes project has an official add-on for this, the **Gateway API Inference Extension**, and GKE offers it as **GKE Inference Gateway**. Projects such as **KServe** and **llm-d** go further. Start simple, and look at these when you run many copies or many models.

There are no silly questions

Q: Can two pods share one GPU?

A: By default, no. A pod that asks for `nvidia.com/gpu: 1` gets a whole GPU. There are ways to share GPUs, but for a model server you usually want the whole GPU and all its memory.

Q: Do I need Kubernetes to self-host?

A: No. You can run vLLM on plain virtual machines or with a managed container service that offers GPUs. Kubernetes makes sense when you already use it, or run several models.

Q: Why not scale to zero at night to save money?

A: You can, if nobody uses Botley at night and the first user in the morning can wait several minutes. For most live agents, keep at least one copy running, ideally two.

Your turn

1. A vLLM pod is stuck in "Pending". The cluster has GPU machines. What is the first thing to check?

2. Kubernetes keeps restarting the vLLM pod while the model is still loading. What do you change?

3. Your autoscaler watches CPU use, and Botley is slow at lunchtime but nothing scales. Why?

4. Sam asks why there are two GPU machines when one is enough for the traffic. What do you say?

The answers are at the back.

Key takeaways

- Pods ask for `nvidia.com/gpu`. GPU machines need the NVIDIA driver and device plugin, and each cloud handles these differently.
- Model servers start in minutes. Keep copies running, keep weights close, and use a readiness probe on `/health`.
- Give vLLM shared memory by mounting a memory volume at `/dev/shm`.
- Scale on vLLM's own metrics (requests waiting, KV cache usage), not CPU or GPU use.
- Always run at least two copies on different machines.
- Smarter inference routing exists. You don't need it on day one.



Chapter eight

Deploying a model for real

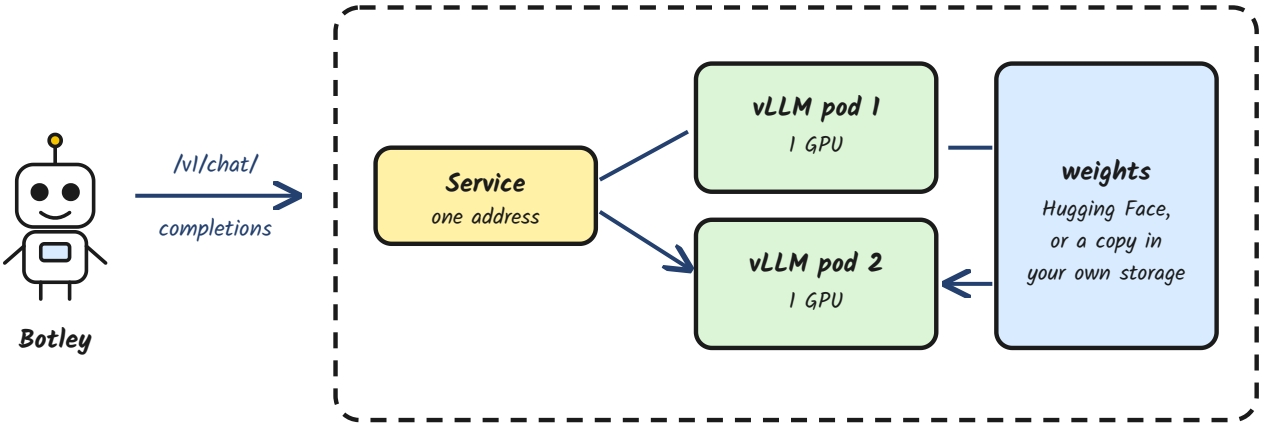
One small open model, two copies, on GKE, EKS or AKS. Then point Botley at it.

Facts in this chapter last checked on 19 October 2026.



> Enough theory. Let's put a model on a GPU and talk to it.

Time to build it. We will serve a small open-weight model with vLLM, run two copies on two GPU machines, and point Botley at it. The goal is to understand each step, so the commands stop looking like magic.



Kubernetes cluster with a GPU node pool

What we are deploying

Choice	What we use	Why
Model	`Qwen/Qwen3-4B-Instruct-2507`	Apache 2.0 license, not gated (no Hugging Face token needed), about 8 GB in BF16.
GPU	NVIDIA L4, 24 GB (A100 on Azure)	Small and cheap. The model fits with plenty of room for the KV cache.
Copies	2	So one machine can fail or be updated without Botley losing its model (Chapter 7).
Serving engine	vLLM, image `vllm/vllm-openai:v0.32.0`	A fixed version, not `latest` (Chapter 6).

> ⚠ **CAREFUL!**
>
> **Before you start:**
>
> - **GPU quota.** New accounts often have no GPU quota (Chapter 4). Ask for enough for two GPU machines in your region, and do it first.
> - **Cost.** GPU machines cost money every hour they run, even when idle. On EKS, two g6.xlarge machines cost about $1.61 per hour on demand, plus $0.10 per hour for the cluster. On GKE Autopilot, GPU pods are billed for the whole machine Autopilot picks, plus a management fee. The Azure recipe's one A100 machine costs about $3.67 per hour. Delete everything when you finish (the last step of each recipe).
> - **Commands change.** These are correct as of October 2026. Check each tool's `--help` and the linked guides before you run them.

Step 1: the parts that are the same everywhere

Create a namespace and a key that Botley must send with every request:

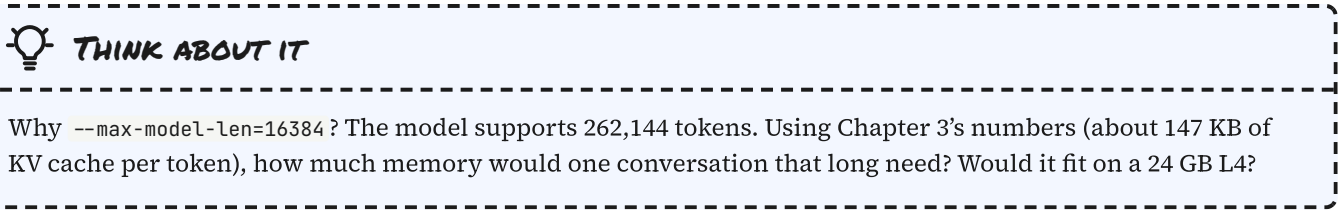
```
kubectl create namespace models
kubectl create secret generic vllm-api-key -n models \
  --from-literal=key="$(openssl rand -hex 24)"
```

Save this file as `vllm.yaml`. The comments explain each part:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm
  namespace: models
spec:
  replicas: 2                       # two copies (Chapter 7)
  selector:
    matchLabels:
      app: vllm
  template:
    metadata:
      labels:
        app: vllm
    spec:
      nodeSelector: {}              # each recipe below tells you what to put here
      containers:
        - name: vllm
          image: vllm/vllm-openai:v0.32.0
          args:
            - --model=Qwen/Qwen3-4B-Instruct-2507
            - --max-model-len=16384          # caps KV cache per chat (Ch. 3)
            - --api-key=$(VLLM_API_KEY)      # filled in from the secret
          env:
            - name: VLLM_API_KEY
              valueFrom:
                secretKeyRef:
                  name: vllm-api-key
                  key: key
          ports:
            - containerPort: 8000
          resources:
            requests:
              cpu: "3"
              memory: 12Gi
              ephemeral-storage: 30Gi        # room for the downloaded weights
            limits:
              nvidia.com/gpu: 1              # one whole GPU for each copy
          volumeMounts:
```

```
            - name: dshm
              mountPath: /dev/shm            # shared memory for vLLM (Chapter 7)
            - name: hf-cache
              mountPath: /root/.cache/huggingface
          startupProbe:                      # allow up to 15 minutes to load
            httpGet:
              path: /health
              port: 8000
            periodSeconds: 10
            failureThreshold: 90
          readinessProbe:                    # only send traffic when ready
            httpGet:
              path: /health
              port: 8000
            periodSeconds: 10
          livenessProbe:                     # restart it if it stops answering
            httpGet:
              path: /health
              port: 8000
            periodSeconds: 30
            failureThreshold: 3
      volumes:
        - name: dshm
          emptyDir:
            medium: Memory
            sizeLimit: 2Gi
        - name: hf-cache
          emptyDir:
            sizeLimit: 25Gi
---
apiVersion: v1
kind: Service
metadata:
  name: vllm
  namespace: models
spec:
  selector:
    app: vllm
  ports:
    - port: 80
      targetPort: 8000
```

This file is based on vLLM's own Kubernetes guide. Because each machine in our recipes has only one GPU, and each copy asks for a whole GPU, the two copies always land on different machines. The weights download from Hugging Face each time a pod starts. Chapter 7 shows faster ways for later.



Step 2: pick your cloud

Recipe 1: Google Cloud (GKE Autopilot)

Autopilot creates the L4 machines and installs the GPU drivers for you.

#	Step	What you type	What happens
1	Create the cluster	`gcloud container clusters create-auto botley-models \` `  --location=us-central1`	Creates an Autopilot cluster. Autopilot adds machines when pods need them.
2	Connect kubectl	`gcloud container clusters get-credentials botley-models \` `  --location=us-central1`	Lets `kubectl` talk to the new cluster.
3	Ask for L4 GPUs	`nodeSelector:` `  cloud.google.com/gke-accelerator: nvidia-l4`	Put this in place of `nodeSelector: {}` in `vllm.yaml`. Autopilot reads it and creates L4 machines.
4	Deploy	`(run Step 1 commands)` `kubectl apply -f vllm.yaml`	Creates the namespace, secret, two vLLM pods and the Service. The first start takes several minutes.
5	Clean up	`gcloud container clusters delete botley-models \` `  --location=us-central1`	Deletes the cluster and its GPU machines.

Official guide: “Serve Gemma open models using GPUs on GKE with vLLM” (docs.cloud.google.com/kubernetes-engine/docs/tutorials/serve-gemma-gpu-vllm). It uses a different model and GPU but the same idea.

Recipe 2: AWS (Amazon EKS)

eksctl creates the cluster and a node group of g6.xlarge machines (1 × L4 each).

#	Step	What you type	What happens

1	Create the cluster	`eksctl create cluster \`
		`  --name botley-models \`
		`  --region us-east-1 \`
		`  --node-type g6.xlarge \`
		`  --nodes 2`
			Creates an EKS cluster with two g6.xlarge machines. For GPU instance types, eksctl picks the EKS-optimized NVIDIA machine image (Amazon Linux 2023) and installs the NVIDIA device plugin for you. It also sets up kubectl.

2	Check the GPUs	`kubectl describe nodes | grep nvidia.com/gpu`	Each node should list `nvidia.com/gpu: 1` under Capacity and Allocatable. If nothing appears, the NVIDIA device plugin is missing. Install it as described in the EKS docs.
3	Ask for those machines	`nodeSelector:` `  node.kubernetes.io/instance-type: g6.xlarge`	Put this in place of `nodeSelector: {}` in `vllm.yaml`.
4	Deploy	`(run Step 1 commands)` `kubectl apply -f vllm.yaml`	Creates everything. The first start takes several minutes.
5	Clean up	`eksctl delete cluster \` `  --name botley-models --region us-east-1`	Deletes the cluster and its machines.

Official guide: “Load & Serve Models on Amazon EKS” (docs.aws.amazon.com/eks/latest/userguide/ml-realtime-inference-llm-inference-vllm.html). It stores the weights in Amazon S3 and streams them into vLLM, which is the faster setup Chapter 7 describes.

Recipe 3: Azure (AKS with the KAITO add-on)

KAITO creates the GPU machines and runs vLLM for you, from one short file.

On Azure, the simplest official path is the **AI toolchain operator** add-on, also called **KAITO**. You don't write the vLLM Deployment yourself. You describe what you want in a small "Workspace" file, and KAITO creates the GPU machine, the vLLM pods and a Service. KAITO only offers its own list of ready-made models ("presets") this way, so this recipe uses Microsoft's Phi-4-mini model on an A100 machine, as in Microsoft's guide.

#	Step	What you type	What happens

1	Create the cluster	`az group create --name botley-rg --location eastus`
		`az aks create --resource-group botley-rg \`
		`  --name botley-models \`
		`  --enable-ai-toolchain-operator \`
		`  --enable-oidc-issuer \`
		`  --generate-ssh-keys`
			Creates a resource group and an AKS cluster with the KAITO add-on turned on.

2	Connect kubectl	`az aks get-credentials \` `  --resource-group botley-rg --name botley-models`	Lets `kubectl` talk to the cluster.

3	Describe the model	`apiVersion: kaito.sh/v1beta1`
		`kind: Workspace`
		`metadata:`
		`  name: workspace-phi-4-mini`
		`resource:`
		`  instanceType: Standard_NC24ads_A100_v4`
		`  labelSelector:`
		`  matchLabels:`
		`  apps: phi-4-mini`
		`inference:`
		`  preset:`
		`  name: phi-4-mini-instruct`
			Save as `workspace.yaml`. It asks for one A100 machine and the Phi-4-mini preset.

4	Deploy and wait	`kubectl apply -f workspace.yaml` `kubectl get workspace workspace-phi-4-mini -w`	KAITO creates the machine and starts vLLM. Wait until the workspace shows it is ready.
5	Clean up	`az group delete --name botley-rg`	Deletes the resource group and everything in it, including the cluster. This can't be undone.

Official guide: "Deploy an AI model on AKS with the AI toolchain operator" (learn.microsoft.com/azure/aks/ai-toolchain-operator). The KAITO project is renaming its presets to full model names, such as `microsoft/Phi-4-mini-instruct`. The add-on on AKS uses the short name `phi-4-mini-instruct` as of October 2026, so check the guide for the name your version expects. If you delete only the workspace, KAITO's GPU machines may keep running, so delete the whole resource group when you are done. To use the same `vllm.yaml` as the other recipes instead, add a GPU node pool to a normal AKS cluster and install the NVIDIA device plugin, as described in Microsoft's "Use GPUs on AKS" guide.

Step 3: check it works

On GKE or EKS, open a tunnel from your laptop to the Service, then ask the model something. (For Azure, see the note after these commands.)

```
kubectl get pods -n models          # wait until both show READY 1/1
kubectl port-forward -n models svc/vllm 8000:80

# in a second terminal
KEY=$(kubectl get secret vllm-api-key -n models -o jsonpath='{.data.key}' | base64 -d)
```

```
curl http://localhost:8000/v1/chat/completions \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "Qwen/Qwen3-4B-Instruct-2507",
       "messages": [{"role": "user",
                     "content": "Suggest one day trip from Lisbon."}]}'
```

You should get an answer in the OpenAI format. Then look at vLLM's numbers (with the tunnel still open):

```
curl -s http://localhost:8000/metrics \
  | grep -E "num_requests_(running|waiting)|kv_cache_usage_perc"
```

On Azure with KAITO, the Service is named after the workspace and listens on port 80, and KAITO sets no API key. Use `kubectl port-forward svc/workspace-phi-4-mini 8000:80`, then send the same curl without the `Authorization` line and with `"model": "phi-4-mini-instruct"`. Because there is no key, keeping this Service private matters even more.

Step 4: point Botley at it

Botley runs in the same cluster or the same private network. Change its two settings (Chapter 6). These are the values for the GKE and EKS recipes. On Azure, use `http://workspace-phi-4-mini.default.svc.cluster.local/v1` and the model name `phi-4-mini-instruct`, with no key.

Setting	Value
Model address	`http://vllm.models.svc.cluster.local/v1` (inside the cluster)
Model name	`Qwen/Qwen3-4B-Instruct-2507`
API key	the key from the `vllm-api-key` secret

Don't give the Service a public address. Only Botley needs to reach it.

⚠ Careful!

This is a starting point, not production yet. Before real users depend on it, add the pieces from Chapter 7: weights stored close by, autoscaling on vLLM's metrics, monitoring and alerts. Then run Botley's real tasks against the new model and compare the results with the old one (Chapter 11).

There are no silly questions

Q: Why does the first start take so long?

A: The cluster may need to create a GPU machine, then pull a large image, then download about 8 GB of weights, then load them into the GPU. Later starts on a machine that already has the image are faster.

Q: Can I use a bigger model with the same file?

A: Yes. Change `--model`, choose GPUs with enough memory (Chapter 4), and set `--tensor-parallel-size` and `nvidia.com/gpu` to the number of GPUs per copy. For a gated model, add a Hugging Face token as a secret and pass it as `HF_TOKEN`.

Q: Why does the Azure recipe use a different model?

A: KAITO's simple path only works with its own list of presets. You can serve other models with KAITO too, or use `vllm.yaml` on AKS as the note says.

Your turn

1. One vLLM pod is ready and the other has been starting for 10 minutes. Name two things you would check.

2. Your curl returns “401 Unauthorized”. What is missing?

3. A teammate wants to expose the vLLM Service on the internet “so testing is easier”. What do you say?

The answers are at the back.

Key takeaways

- The same `vllm.yaml` works on GKE and EKS. Only the node selector changes.
- On GKE Autopilot, a node selector for `nvidia-l4` is enough to get L4 machines with drivers.
- On EKS, check that each node shows an allocatable `nvidia.com/gpu`.
- On AKS, the KAITO add-on deploys preset models with vLLM from one short file.
- Use a fixed vLLM version, an API key, a long startup probe and shared memory.
- Keep the model private, test with port-forward, then point Botley at the in-cluster address.



Chapter nine

Does it pay off?

Two kinds of bill, one worked example with real prices, and the costs people forget.

Facts in this chapter last checked on 19 October 2026.



Just tell me: is our own GPU cheaper or not?

Sam's first question was about money. The short answer: it depends on how busy you are. This chapter shows how to work it out, with real prices, so you can do the math for your own agent.

Two kinds of bill

- **Pay per token:** the bill grows with use. No use, no bill.
- **Your own GPU:** the bill is the same every hour the GPU is on, whether it is busy or idle.

So the question is: **how busy does the GPU need to be before it is cheaper?** That point is called the **break-even**.

A worked example

Let's compare an 8B model on one GPU with the same model on Amazon Bedrock.

Step 1: the GPU bill. One AWS g6e.xlarge machine (1 × L40S GPU) costs $1.861 per hour on demand in us-east-1. A month has about 730 hours:

```
$1.861 × 730 hours = $1,358.53 per month
```

Step 2: how much the GPU can do. We need a measured number. NVIDIA published one: Llama 3.1 8B in FP8 on one L40S, with 100 users at once and 1,000 tokens in and 1,000 tokens out per request, wrote about 2,869 output tokens per second. If the GPU worked like that all month:

```
2,869 tokens/second × 3,600 seconds × 730 hours ≈ 7.54 billion output tokens
```

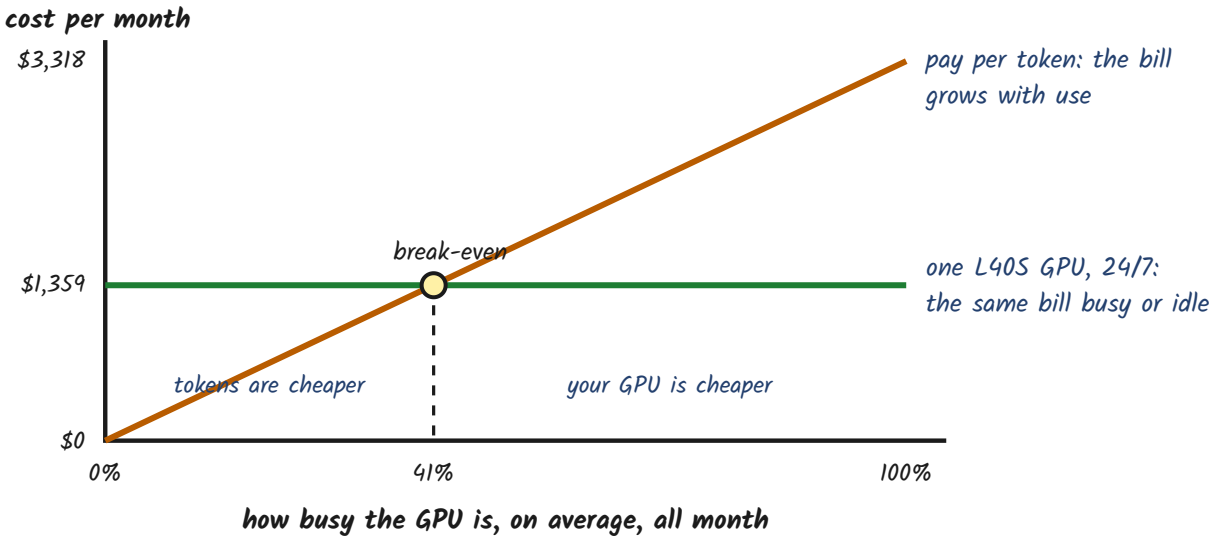
Each request also had 1,000 input tokens, so that is about 7.54 billion input tokens too.

Step 3: the same tokens on Bedrock. Llama 3.1 8B Instruct on Bedrock costs $0.22 per million input tokens and $0.22 per million output tokens:

```
15.08 billion tokens × $0.22 per million ≈ $3,318 per month
```

Step 4: the break-even.

```
$1,358.53 ÷ $3,318 ≈ 41%
```



If the GPU is busy like this **more than about 41% of the time, all month, day and night**, the GPU is cheaper. Below that, paying per token is cheaper.

Sources: AWS EC2 on-demand pricing (g6e.xlarge, us-east-1), the AWS Price List for Amazon Bedrock (us-east-1), and NVIDIA NIM LLM benchmarks (NIM 1.8.0). All checked 19 October 2026. The throughput number comes from NVIDIA's own serving software, not vLLM. Your numbers will be different. Measure your own model, with your own request lengths, before you decide.

What changes the answer

Small changes move the break-even a lot. Each line changes one thing in the example above:

Change	Break-even
The example as it is	about 41% busy
BF16 instead of FP8 (NVIDIA measured about 1,613 tokens per second)	about 73% busy
Two GPUs for safety (Chapter 7)	traffic of about 82% of one GPU's full speed (each GPU about 41% busy)
Compare with a cheaper managed model, Ministral 8B at $0.15 per million	about 60% busy
Compare with Amazon Nova Micro ($0.035 in, $0.14 out per million)	over 100%: the GPU never wins

THINK ABOUT IT

Most agents are busy in office hours and quiet at night. Botley's traffic is high from 9 to 6 and almost nothing after that. Could its GPU really be 41% busy on average over a whole month?

CAREFUL!

"41% busy" means 41% of full speed, on average, every hour of the month. Full speed here is 100 users at once, all the time. A GPU that is packed at lunchtime and empty at night may average well under that. Look at your real traffic over a whole week, not your busiest hour.

The costs people forget

The GPU price is only part of the bill. AWS's own guidance for generative AI says that "more often than not, it is beneficial to opt for a managed or serverless hosting paradigm", because the total cost of hosting models yourself is so hard to pin down. Here is what that total includes:

Hidden cost	Why it matters
People	Someone sets up, upgrades and fixes the GPU cluster, and is on call when it breaks at night.
The second copy	You need at least two copies for safety (Chapter 7). That doubles the GPU bill.
Idle hours	Nights, weekends and holidays cost the same as busy hours, unless you scale down.
Getting GPUs	Quota requests, and popular GPUs that are sold out in your region.
Upgrades	vLLM releases about every two weeks. New models come out often. Each change needs testing.
Storage and network	Weights, images, logs and data moving between services.
Testing models	Checking that the open model is good enough for your tasks, and checking again after each change.

Ways to bring the GPU bill down

- **Commit for longer.** AWS Savings Plans (up to 72% off), Google committed use discounts (about 55% off for three years on g2 machines) and Azure reservations (up to 72% off) lower the hourly price, if you are sure you will use the GPU for that long.
- **Use Spot for extra capacity.** Up to about 90% off, but the cloud can take the machine back at short notice. Use it for extra copies or batch jobs, not your only copy.
- **Use a smaller or quantized model.** Less memory, more conversations per GPU (Chapter 4).
- **Scale down when it is quiet,** if your users can accept a slow first answer after a quiet period.

Is our own GPU cheaper?

Sam	Maya
The GPU is $1,359 a month. Our token bill was higher than that last month. Easy decision!	
	That's one GPU. We need two for safety, so $2,717. Plus the time I spend running them.
But we'd be paying the same every month. No more surprises.	
	True, and that's worth something. But our traffic is mostly office hours. Let's measure a full week before we decide.
Fine. Show me the numbers on Friday.	

There are no silly questions

Q: Why didn't you use an L4 for the example? It's cheaper.

A: We could not find a reliable published throughput number for an 8B model on one L4. Without a measured number, any break-even would be a guess. Run a load test on your own model and GPU, then use the same four steps.

Q: Does the break-even get better with bigger GPUs?

A: Not by itself. A bigger GPU costs more per hour and can do more work per hour. What matters is how busy you keep it. Bigger models also have higher per-token prices on managed services, so do the math for each case.

Q: Isn't reserved capacity on a managed service the same trade-off?

A: Yes. Reserved capacity is also paid by the hour, busy or not. The difference is that the provider runs the GPUs, so the "people" line in the hidden costs is much smaller.

Your turn

Use the four steps. (The throughput number in question 2 is made up for practice. Measure your own for real decisions.)

1. One AWS g6.xlarge (1 × L4) costs $0.8048 per hour. What does it cost for a 730-hour month?

2. Suppose you measure that this L4 writes 600 output tokens per second at full use, with the same number of input tokens. How many tokens in total could it handle in a month?

3. What would those tokens cost on Bedrock at $0.22 per million in and out?

4. What is the break-even? Would you switch if your GPU would be about 30% busy?

The answers are at the back.

Key takeaways

- Pay-per-token bills grow with use. GPU bills are the same every hour the GPU is on.
- Break-even = GPU cost per month ÷ what a month of the GPU's full output would cost on the managed service.
- In our example with real prices, one L40S beats Bedrock above about 41% average use. With two GPUs for safety, you need about twice the traffic: about 82% of one GPU's full speed.
- Very cheap managed models (like Nova Micro) can beat your own GPU at any level of use.
- Count the hidden costs: people, the second copy, idle hours, upgrades and testing.
- Use your own measured throughput and a full week of real traffic before deciding.



Chapter ten

Reasons that aren't money

Privacy, control, custom models and more. Each one is real, and each one has a "but".

Facts in this chapter last checked on 19 October 2026.



Forget the bill for a second. Are there other reasons to run our own?

Money is not the only reason teams run their own models. Here are the other common reasons. For each one, there is also a fair counter-point, because the managed services have caught up on many of them.

Reason 1: data must stay on machines you control

Sam's second question: the customer wants their data to stay inside the company's own cloud account. With your own GPUs, prompts and answers never leave machines that you run.

But: check what the rule actually requires. Many rules say "don't let the provider train on our data" or "keep data in the EU". The managed services can meet those: they promise not to train on your data without permission, and you can choose the region, or a data zone on Microsoft Foundry (Chapter 5). If the rule really says "only on machines we control", then self-hosting, or the provider's dedicated options, may be the answer.

Reason 2: you want to keep a model version for as long as you like

On a managed service, the provider retires models on its schedule. On Bedrock, requests to a model fail after its End-of-Life date. On Microsoft Foundry, generally available models usually retire 18 months after launch, and the dates can't be extended (Chapter 5). If you have carefully tested Botley with one exact model, a forced change means testing again.

With your own copy of the weights, you can run the same model for as long as you like.

But: "never changing" is not free. You still need to update vLLM, drivers and machine images for security fixes, and test after each update. And you miss out on newer, better models unless you choose to move.

Reason 3: you have your own fine-tuned model

Fine-tuning means training an existing model a little more on your own examples, so it gets better at your task. If you have a fine-tuned open model, you need somewhere to run it.

But: you don't always need your own GPUs for this. Bedrock Custom Model Import can run imported weights for several model families, Google's Agent Platform can deploy open models from Model Garden on dedicated machines, and Microsoft Foundry has managed compute (Chapter 5). Each still charges by the hour or minute while the model runs.

Reason 4: it must work without the internet

Some places can't send anything outside: a factory floor, a ship, some government and defense systems. They need the model on their own machines, with no connection to a model service. For these, self-hosting is almost always the answer. (One exception: Google offers Gemini on its Google Distributed Cloud hardware, installed at your own site.)

Reason 5: control over speed

With your own GPUs, you decide how many users share a GPU (Chapter 3). You can trade throughput for faster answers, put the model next to Botley in the same network, and avoid sharing with other customers.

But: managed services also offer faster tiers (such as Bedrock's Priority tier) and reserved capacity that is yours alone. And keeping fast answers on your own GPUs means paying for spare capacity at busy times.

Reason 6: no limits set by someone else

Managed services have rate limits and quotas per account. Your own GPU has no outside limit.

But: it has its own limit: how much the GPU can do. When it is full, requests wait in vLLM's queue until you add more GPUs, and adding GPUs takes minutes (Chapter 7).

The reasons side by side

Reason	Does self-hosting help?	Can a managed option help too?
Data stays in a region	Yes	Yes: pick the region or data zone
Data only on machines you control	Yes	Partly: dedicated options run on machines only you use, but the provider runs them
Keep one model version for a long time	Yes	No: the provider decides when a version retires
Run your own fine-tuned model	Yes	Yes, for supported model families
Work with no internet at all	Yes	No
Control over speed	Yes	Partly: faster tiers and reserved capacity
No outside rate limits	Yes, but your GPU has its own limit	Partly: reserved capacity

Privacy: own GPUs or managed?

Customer's security team	Maya
We don't want our data used to train anyone's model.	All three managed services promise that in writing. Here is the page for the one we use.
We also need it processed in the EU.	We can choose an EU region, or an EU data zone. We just change a setting.
And nothing may leave infrastructure your company controls.	That one the managed service can't meet. For that, we'd run the model ourselves.

There are no silly questions

Q: Is self-hosting automatically more secure?

A: No. It moves the job to your team. You must patch the machines, protect the vLLM endpoint (it has almost no security of its own, Chapter 6) and watch the logs. A well-run managed service can be safer than a badly run cluster.

Q: Can we self-host just for the one customer who needs it?

A: Yes. The agent can call different models for different customers. Botley could use a managed model for most users and your own model for the customer with strict rules.

Your turn

Does each customer need self-hosting? Answer **yes**, **no** or **maybe**, and say why.

1. "Our data must not be used to train AI models."
2. "All processing must stay in the EU."
3. "The system runs on an oil platform with no internet connection."
4. "We need the exact same model version for at least three years, for audit reasons."

The answers are at the back.

Key takeaways

- Good non-money reasons: data only on machines you control, keeping one model version, fine-tuned models, no internet, control over speed.
- Many privacy rules can be met by managed services: no training on your data, and a choice of region or data zone.
- Only self-hosting lets you keep a model version for as long as you like, or run with no internet at all.
- Fine-tuned models can also run on managed dedicated options.
- Self-hosting is not automatically more secure. It moves the work to your team.

Chapter eleven

11 Choosing your path

A decision map, mixing both paths, how to test a new model, and a checklist for launch day.

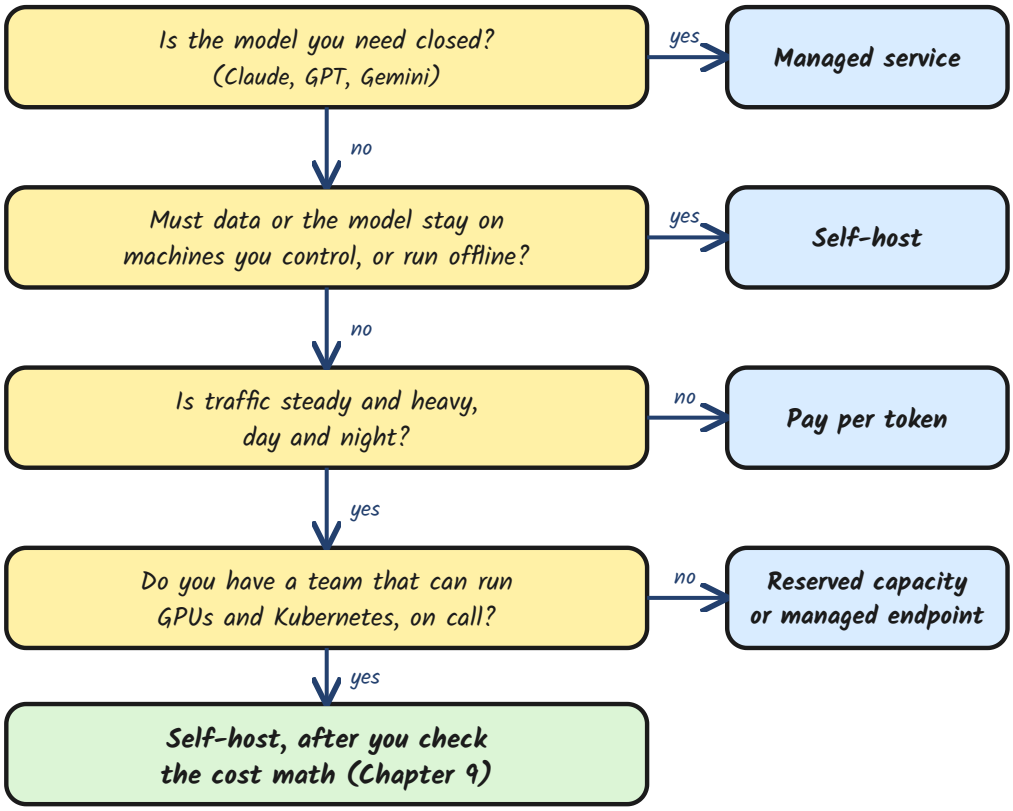
Facts in this chapter last checked on 19 October 2026.



> Restaurant, private room or my own kitchen? Let's decide.

You now know how models use GPUs, what the managed services offer, how to run your own, and how to do the math. Here is how to put it together.

The decision map



Start at the top. Most new agents stop at the first or third box.

The order matters:

1. **Closed model?** If the model you need is Claude, GPT or Gemini, the decision is made: use a managed service.
2. **A hard rule about where data or the model may run?** If it must be on machines you control, or offline, self-hosting is the answer (Chapter 10).
3. **Steady, heavy traffic?** If not, pay per token. You won't keep a GPU busy enough (Chapter 9).
4. **A team that can run GPUs?** If not, use reserved capacity or a managed open-model endpoint (Chapter 5). If yes, self-host, after you check the math.

You can mix both paths

You don't have to pick only one. Because vLLM and most model services accept the same style of request, an agent can call different models for different jobs. This is often called **model routing**.

Easy, frequent jobs → small model on your own GPU

- *Sorting incoming messages*
- *Pulling dates and places out of text*
- *Short summaries*

Hard, rare jobs → large managed model

- *Planning a complex trip with many steps*
- *Unusual questions*
- *Anything the small model gets wrong*

This keeps your GPU busy with the high-volume work, which helps the break-even, while the hard cases still get the best model.

Test before you switch

A new model will behave differently from the old one (Chapter 1). Don't switch Botley on a feeling. Test it:

1. **Collect real tasks.** Take a set of real past conversations, for example 50 to 200, with what a good answer looks like. Include the hard ones and the tool calls.
2. **Run both models** on the same tasks, with the same instructions.
3. **Compare what matters for Botley:** Did it call the right tool with the right details? Is the answer correct? Is it fast enough? What did it cost?
4. **Test the load.** Send many requests at once and watch time to first token and requests waiting (Chapter 3). vLLM includes a benchmark tool, `vllm bench serve`, for this.
5. **Switch a small share first.** Send a small share of real users to the new model, watch it, then increase it.

> ⚠ **CAREFUL!**
>
> **Test again after every change.** A new model version, a new quantization, or a new vLLM version can all change answers. Keep your test set and run it each time.

What Maya told Sam

Question 1: Would our own model be cheaper?

Not yet. Our traffic is mostly office hours. With two GPUs for safety, we'd need traffic of about 82% of one GPU's full speed, day and night, to beat paying per token. We are nowhere near that. We'll measure again every quarter.

Question 2: Can the customer's data stay inside our cloud account?

Their real rule is "EU only, and no training on our data". The managed service meets both with the right region setting. If a customer ever needs "only on our own machines", we can run a small open model for that customer alone.

Next step: *build a test set of 100 real Botley tasks, so we are ready to compare models when we need to.*

Launch-day checklist for a self-hosted model

- [ ] The license of the exact model version allows your use (Chapter 2)
- [ ] Weights and KV cache fit, with `--max-model-len` set on purpose (Chapters 3 and 4)
- [ ] GPU quota approved, in the region you need (Chapter 4)
- [ ] A fixed vLLM version, not `latest` (Chapter 6)
- [ ] The vLLM endpoint is private, with an API key (Chapter 6)
- [ ] At least two copies, on different machines (Chapter 7)

- [ ] Startup and readiness probes on `/health`, and shared memory at `/dev/shm` (Chapters 7 and 8)
- [ ] Weights stored close to the cluster, not downloaded from the internet on every start (Chapter 7)
- [ ] Autoscaling on vLLM's own metrics, with alerts on requests waiting and time to first token (Chapter 7)
- [ ] The cost math done with your own measured throughput and a week of real traffic (Chapter 9)
- [ ] Botley's real tasks tested on the new model, with a plan to switch users over slowly (this chapter)
- [ ] Someone is on call for the GPUs, and knows how to switch Botley back to a managed model

There are no silly questions

Q: What if we choose wrong?

A: Because Botley only changes two settings to move between vLLM and a managed service, you can switch back. Keep the managed option set up and tested, so moving back only means changing two settings.

Q: How often should we check the decision again?

A: When traffic changes a lot, when prices change, or when a new model comes out that could do the job with a smaller GPU. For most teams, every few months is enough.

Your turn

Use the decision map. Which path would you choose?

1. A start-up has an agent that uses Claude, and its traffic is growing fast.

2. A hospital wants an assistant that runs only on its own servers, which have no internet connection.

3. A shop's support agent gets steady traffic around the clock, uses an open model, and has no one who knows Kubernetes.

4. A large company with a platform team already runs GKE, and has heavy, steady traffic for an open model. The math shows the GPUs would be about 85% busy.

The answers are at the back.

Key takeaways

- Start with the hard limits: closed models need a managed service, and strict "our machines only" rules need self-hosting.
- Then look at traffic. Low or uneven traffic means pay per token.
- Self-host only with steady, heavy traffic, a team to run it, and math that works.
- You can mix: small self-hosted models for easy, frequent jobs, and large managed models for hard ones.
- Test a new model on real tasks before switching, and after every change.
- Keep the managed option ready, so you can switch back with a settings change.

Word list and answers

Every tricky word in this notebook, in plain English, and the answers to the exercises

Word	What it means
LLM	Large language model. The part of an AI agent that reads and writes text.
Inference	Running a model to answer requests. The model's weights don't change.
Weights, parameter	The numbers a model is made of. Each single number is a parameter. "8B" means about 8 billion parameters.
Closed model	A model whose weights are never given out, such as Claude, GPT or Gemini. Only available through an API.
Open-weight model	A model whose weights anyone can download and run, such as Llama, Qwen, Gemma, Mistral, gpt-oss or DeepSeek.
License	The rules for what you may do with a model. Apache 2.0 and MIT allow commercial use. Some model makers use their own licenses.
Hugging Face	A website where most open-weight models are shared.
Model card	The page that describes a model: what it is for, how to run it, and its license.
Gated model	A model you can only download after accepting its license and using an access token.
Precision: BF16, FP8, INT4	How many bits each number uses: 16, 8 or 4. Fewer bits means less memory.
Quantization	Storing a model's numbers with fewer bits, to save memory. It can lower quality, so test it.
Mixture of experts (MoE)	A model that uses only part of its weights for each token. All the weights must still be in memory.
Token, tokenizer	A token is a small piece of text. The tokenizer cuts text into tokens and joins them back.
Prefill	Stage 1 of an answer: reading the whole request at once.
Decode	Stage 2 of an answer: writing it one token at a time.
KV cache	Notes the model keeps about every token in each conversation, so it doesn't redo work. It lives in GPU memory.
Batching, continuous batching	Working on many conversations in one step. Continuous batching adds new ones as soon as there is room.
PagedAttention	vLLM's way of storing the KV cache in small blocks, so little memory is wasted.
Time to first token	How long a user waits before the answer starts.
Throughput	Tokens per second for all users together.
GPU, GPU memory	A chip that does many simple calculations at once. Its own memory must hold the weights and the KV cache.
Tensor parallelism	Splitting one model across several GPUs in the same machine.
TPU, Inferentia, Trainium	Chips made by Google (TPU) and Amazon (Inferentia, Trainium). They need software built for them.
Quota	The cloud's limit on how many machines of a kind you may run. GPU quota often starts at zero.
On-demand, Spot	On-demand machines are yours until you stop them. Spot machines are much cheaper but can be taken back at short notice.

Word	What it means
Pay per token	Paying a model service for each token in and out.
Batch inference	Sending many requests to a model service at once and collecting answers later, at a lower price.
Reserved capacity	Paying a model service by the hour for capacity only you use: Provisioned Throughput on Bedrock and Agent Platform, PTUs on Foundry.
Data zone	On Microsoft Foundry, a deployment that keeps processing inside a zone such as the US or the EU.
Model retirement	When a model service switches off an old model version. Requests to it then fail.
Serving engine, vLLM	Software that loads a model into a GPU and answers requests. vLLM is one of the most widely used open-source ones.
OpenAI-compatible API	Accepting requests in the same format as OpenAI's API, such as `/v1/chat/completions`.
`/health`, `/metrics`	vLLM's addresses for "am I ready?" and for its numbers in Prometheus format.
GPU node pool	A group of machines with GPUs in a Kubernetes cluster.
Device plugin, GPU Operator	The device plugin tells Kubernetes how many GPUs a machine has. NVIDIA's GPU Operator installs it and the other GPU parts.
Startup and readiness probes	Checks that tell Kubernetes when a container has started and when it is ready for traffic.
`/dev/shm`	Shared memory inside a container. vLLM needs more than the default.
KEDA, Karpenter	KEDA adds pods based on numbers such as a Prometheus query. Karpenter adds machines on EKS.
KAITO	Azure's AI toolchain operator. An AKS add-on that deploys preset models with vLLM.
Fine-tuning	Training an existing model a little more on your own examples.
LoRA adapter	A small add-on that changes how a model behaves, without changing the model's own weights.
Break-even	The level of use at which your own GPU costs the same as paying per token.
Model routing	Sending different jobs to different models, for example easy ones to a small model and hard ones to a large one.

Answers

Chapter 1, Your turn: which way is it?

#	Answer	Why
1	Pay per token	A price per million tokens is the restaurant.
2	Your own GPUs	Maya's team runs the model and the GPUs.
3	Reserved capacity	Paying by the hour for PTUs, busy or not, is the private room.
4	Not possible	Gemini is a closed model. You can't download it.

Chapter 2, Your turn: how big are the weights?

1. 14 billion × 2 bytes = about **28 GB**.
2. 70 billion × 1 byte = about **70 GB**.
3. 8 billion × 0.5 bytes = **a bit over 4 GB**.
4. About **60 GB** (30 billion × 2 bytes). Only 3 billion are used for each token, but all 30 billion must be in memory.

Chapter 3, Think about it: which stage takes Botley's GPU time?

Probably stage 1. Botley sends thousands of tokens and gets a short reply, so reading the request is a large share of the work. That is also why the wait before the first word matters so much for Botley.

Chapter 3, Your turn

1. Stage 1, reading the request (prefill).
2. Stage 2, writing the answer (decode).
3. 20 × 4,096 tokens × about 147 KB = about **12 GB**.
4. The GPU can write tokens for many conversations in one step. With one user, most of it sits idle, but you still pay for all of it. NVIDIA's example wrote about 71 tokens per second for one user, and about 2,869 for 100 users, on the same GPU.

Chapter 4, Your turn: will it fit?

1. **Yes.** About 16 GB of weights, out of about 21.6 GB usable. About 5.6 GB is left for the KV cache.
2. **No.** About 140 GB of weights won't fit in 80 GB. Use FP8 on two or more GPUs, or a bigger GPU.
3. **Yes.** About 70 GB of weights, out of about 127 GB usable, leaves about 57 GB for the KV cache.
4. **Yes.** About 8 GB of weights, out of about 21.6 GB usable, leaves about 13.6 GB for the KV cache. (With vLLM's default of 92%, it is about 14 GB.)

Chapter 5, Your turn: choose the option

#	Answer	Why
1	Batch	Nobody needs the answers right away, and batch costs 50% less.
2	Pay per token	Low traffic, mostly in office hours. Nights and weekends cost nothing.
3	An open model on GPUs only you use	For example Bedrock Custom Model Import, Agent Platform self-deployed models, or Foundry managed compute. No Kubernetes to run.
4	Reserved capacity	Steady traffic all day, and speed that doesn't depend on other customers.

Chapter 6, Your turn

1. `--max-model-len`. Check it is long enough for Botley's longest conversations, and that the KV cache can hold them.
2. `vllm:num_requests_waiting`.
3. Ollama is made for one computer. By default it only listens on that computer and handles one request at a time. Use vLLM for many users.
4. vLLM accepts requests in the same OpenAI format that Botley already uses. Only the address and the model name change.

Chapter 7, Your turn

1. Run `kubectl describe node` and check that it lists `nvidia.com/gpu` under Allocatable. If not, the driver or the NVIDIA device plugin is missing. Also check that a GPU is free: each copy takes a whole GPU.
2. Give it a startup probe on `/health` that allows enough time to load the model, so Kubernetes doesn't restart it too early.
3. CPU use stays low because the work happens on the GPU. Scale on vLLM's own metrics, such as requests waiting.
4. With one copy, any machine failure or update takes Botley's model away for several minutes while a new GPU machine starts and loads the model.

Chapter 8, Think about it: why —max-model-len=16384?

262,144 tokens × about 147 KB is about **38.7 GB** of KV cache for a single conversation. That is more than the whole 24 GB L4. vLLM checks this when it starts and stops with an error if even one conversation of the maximum length can't fit. Setting 16,384 tokens (about 2.4 GB each) leaves room for about five full-length conversations, or many shorter ones.

Chapter 8, Your turn

1. Run `kubectl describe pod` and look at the events: is it waiting for a GPU machine, or for quota? Then run `kubectl logs` to see if it is still downloading or loading the weights.
2. The API key. Send `Authorization: Bearer` with the key from the `vllm-api-key` secret.
3. No. vLLM has almost no security of its own (Chapter 6). Use `kubectl port-forward` for testing, and keep the Service private.

Chapter 9, Your turn

1. $0.8048 × 730 = about **$587.50** per month.
2. 600 × 3,600 × 730 = about 1.58 billion output tokens, plus the same number of input tokens: about **3.15 billion tokens** in total.
3. 3,153.6 million × $0.22 per million = about **$693.79**.
4. $587.50 ÷ $693.79 = about **85%**. At 30% busy, no: paying per token is clearly cheaper. And that is before adding a second GPU and your team's time.

Chapter 10, Your turn

1. **No.** All three managed services promise not to train on your data without permission.
2. **No.** Choose an EU region, or an EU data zone on Foundry. Check the model is offered there.
3. **Yes.** With no internet, the model must run on machines on the platform.
4. **Yes, or at least keep the weights yourself.** Managed services retire model versions on their own schedule. On Foundry, retirement dates can't be extended.

Chapter 11, Your turn

1. **A managed service.** Claude is a closed model. Pay per token while traffic grows.
2. **Self-host.** The servers have no internet connection, so a managed service can't be used.
3. **Reserved capacity or a managed open-model endpoint.** The traffic is steady, but nobody can run Kubernetes.
4. **Self-host.** A platform team, steady heavy traffic, and math that shows the GPUs would be busy enough.